Common Access Token

CTA-5007-B

April 2025

FORWARD

This document was developed by the Web Application Video Ecosystem (WAVE) Project of the Consumer Technology Association. The WAVE Project is a broad industry initiative of content, technology, infrastructure and device companies, all working together towards commercial Internet video interoperability based on industry standards.

(This page intentionally left blank.)

TABLE OF CONTENTS

FIGURES

(This page intentionally left blank.)

Common Access Token

1 INTRODUCTION

This document defines a Common Access Token (CAT): a simple, extensible, policy-bearing bearer token for content access. The primary use case for this token is to allow content providers to enforce access policies efficiently, flexibly, and interoperably. This is particularly valuable for audiovisual content access control; however, it is equally beneficial as a general-purpose bearer token for any content type. This token is usable as an OAUTH bearer token, a URI signing token, or more generally as a mechanism for conveying delivery policy.

2 LICENSE

3 REFERENCES

1. IETF RFC 8392, *CBOR Web Token (CWT)*, https://tools.ietf.org/html/rfc8392.
2. IETF RFC 4648, *The Base16, Base32, and Base64 Data Encodings*, https://tools.ietf.org/html/rfc4648.
3. IETF RFC 9053, *CBOR Object Signing and Encryption (COSE): Initial Algorithms*, https://tools.ietf.org/html/rfc9053.
4. IETF RFC 8230, *Using RSA Algorithms with CBOR Object Signing and Encryption (COSE) Messages*, https://tools.ietf.org/html/rfc8230.

5. IETF RFC 8610, *Concise Data Definition Language (CDDL): A Notational Convention to Express Concise Binary Object Representation (CBOR) and JSON Data Structures*, https://tools.ietf.org/html/rfc8610.
6. IETF RFC 6570, *URI Template*, https://tools.ietf.org/html/rfc6570.
7. IETF RFC 9449, *OAuth 2.0 Demonstrating Proof of Possession (DPoP)*, https://tools.ietf.org/html/rfc9449.
8. IETF RFC 6750, *The OAuth 2.0 Authorization Framework: Bearer Token Usage*, https://tools.ietf.org/html/rfc6750.
9. IETF RFC 8949, *Concise Binary Object Representation (CBOR)*, https://tools.ietf.org/html/rfc8949.
10. IETF RFC 9164, *Concise Binary Object Representation (CBOR) Tags for IPv4 and IPv6 Addresses and Prefixes*, https://tools.ietf.org/html/rfc9164.
11. IETF RFC 3986, *Uniform Resource Identifier (URI): Generic Syntax*, https://tools.ietf.org/html/rfc3986.
12. IETF RFC 9110, *HTTP Semantics*, https://tools.ietf.org/html/rfc9110.
13. IETF RFC 5280, *Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile*, https://tools.ietf.org/html/rfc5280.
14. IETF RFC 8747, *Proof-of-Possession Key Semantics for CBOR Web Tokens (CWTs)*, https://tools.ietf.org/html/rfc8747.
15. IETF RFC 7638, *JSON Web Key (JWK) Thumbprint*, https://tools.ietf.org/html/rfc7638.
16. IETF RFC 9052, *CBOR Object Signing and Encryption (COSE): Structures and Process*, https://tools.ietf.org/html/rfc9052.
17. IETF RFC 6265, *HTTP State Management Mechanism*, https://tools.ietf.org/html/rfc6265.
18. IETF RFC 8941, *Structured Field Values for HTTP*, https://tools.ietf.org/html/rfc8941.
19. ETSI TS 104 002 V1.1.1, *DASH-IF Forensic A/B Watermarking: An interoperable watermarking integration schema*, https://www.etsi.org/deliver/etsi_ts/104000_104099/104002/01.01.01_60/ts_104002v010101p.pdf.
20. DMA TR 8350.2, *Department of Defense World Geodetic System 1984, Its Definition and Relationships with Local Geodetic Systems*, https://apps.dtic.mil/sti/pdfs/ADA280358.pdf.
21. CTA-5009-A, *Fast and Readable Geographical Hashing*, https://shop.cta.tech/products/fast-and-readable-geographical-hashing-cta-5009.
22. IETF RFC 9679, *CBOR Object Signing and Encryption (COSE) Key Thumbprint*, https://tools.ietf.org/html/rfc9679.
23. FIPS-180-4, *Secure Hash Standard (SHS)*, https://csrc.nist.gov/pubs/fips/180-4/upd1/final.

3.1 Draft references

CTA-5007 examples sometimes also reference the contents of a draft document that remains a work-in-progress:

24. draft-lemmons-composite-claims, *Composite Token Claims*, https://datatracker.ietf.org/doc/draft-lemmons-composite-claims/.

The "and", "or", "nor", "crit", and "env" claims are proposed in this reference. This document does not define or use these claims. Those claims do not have assigned claim keys and cannot be used. However, many of the claims defined herein are designed to work well with the concepts they provide, and so the examples remain instructive. The format and functionality of these claims is illustrative only and does not indicate anything about how these claims may ultimately function.

Examples that use these claims will be encoded with an invalid claim key. As a result, decoders may reject these improper example tokens.

It is the express intention that this document be updated in the future to create compatibility with these claims if they are published.

4 ACCESS TOKEN

4.1 Introduction

This section defines the format of the access token, how the token is conveyed and how the token is verified by a recipient.

4.2 Requirements

The following provides some requirements on the elements the token shall protect when added to requests:

- The request network sources. The token shall allow access to multiple sources and blocks of IPs for use in situations where a token-provider knows multiple IPs for the client or wishes to allow access from a larger network. The IP allowed to use a token should be flexible enough to cover multiple IP addresses and IP blocks.
- The entire URI being accessed. The token shall be able to specify a regular language that describes a set of URIs to accept. The evaluation of such a language should not be exponential and should be fast in the usual cases. As an example, regexes work well.
- The methods allowed for access. The token shall be able, for example, to allow "GET" and "OPTIONS", but prohibit "POST" and "PUT". It can take the form of a claim that lists allowable and non-allowable methods.
- The headers and their content. The token shall be able to allow or deny access based on request headers. For example, a token-provider might wish to allow some, but not all, clients to use the Range header, or to use it with specific values. The idea is to protect specific headers. Most headers will not be useful to protect, but there are specific times one might want to deny access to certain kinds of requests.
- The scheme. The scheme is part of the URI, therefore a token that protects the URI shall allow schemes to be specified as well. There is a need for some extensibility to add some query string

parameters to the original URL. For example, a carefully written regex should be able to allow flexibility in parts of the query string.

- Usage via a nonce. While not appropriate in all circumstances, some use cases warrant a single-use nonce to prevent or reduce replay. This is a situation where the nonce is provided by the signer. It is put in the signed token and redeemed once.

In addition, the token should adhere to the following:

- The token should be as terse as is reasonable. That does not need to be massively compressed, but since this is probably going into every single request, adding half a megabyte to the request line is undesirable.
- The token should be transportable, ideally in the same form, in multiple locations: path, query, header, cookie, and so on for whatever makes sense.
- The token should not contain extraneous personal information. Personal details about the owner of the token should generally be avoided: Name, Zip Code, Age, Street Address, and so on.
- The token should use flexible cryptography. As examples, JWT and CWT are especially good at this, because the set of recommended and allowed encryption schemes is actively maintained and third-party libraries are pretty good about having options.
- The token should be in reasonably standard formats. Reasonably available libraries to parse and process it should be available.
- A client that understands the token should be able to know what it provides access to. It might not be able to validate it, especially if a symmetric key is used for signing, but it should generally be able to inspect the token for whatever their purposes are. Private data should be avoided.

For its claims, there is a desire to control geographical location, and it needs all the standard expiration, not-before, and other timing-related limitations.

4.3 Token definitions

This document defines one encoding: The token MUST be encoded as a CBOR Web Token (CWT) [RFC 8392].

4.3.1 Encodings

4.3.1.1 CWT Encoding

The token MUST be created and validated according to the process defined in RFC 8392 Section 7 to create a Signed, MACed, or encrypted CWT. After creating, it MUST be base64url-encoded according to the RFC 4648 Section 5. Before validation, it MUST be base64url-decoded.

Padding as described in RFC 4648 Section 3.2 MAY be omitted. CATs are terminated unambiguously in all contexts and their lengths can be properly determined. Omitting padding makes their use in path and query parameters less prone to parsing errors on account of using equal signs ('=') as padding characters. Recipients MUST properly parse tokens that do not have padding characters on the end and also tokens that do have them.

Recipients MUST process tokens that are 4096 or fewer bytes but SHOULD process tokens of any length that does not impose an excessive resource burden. If multiple tokens are present, they MUST all be processed if they total to fewer than 4096 bytes. If too many bytes worth of tokens are present,

recipients MAY treat some or all of the tokens as though they are not present, which is likely to result in a denial of access.

4.3.1.1.1 Algorithms

Recipients MUST support the HMAC 256/256 (kty number 5, RFC 9053 Section 3.1), PS256 (kty number -37, RFC 8230 Section 2), and ES256 (kty number -7, RFC 9053 Section 2.1) algorithms. They SHOULD accept a CWT Signed or MACed with any algorithm they support. No MANDATORY algorithms are defined in this document for encrypted CWTs. Recipients MAY choose not to support encrypted CWTs.

Recipients processing a CAT MUST support the ES256 algorithm for jkt confirmations but SHOULD process a jkt confirmation using any algorithm they support.

The "kid" and "alg" header fields are OPTIONAL in a COSE header and they are OPTIONAL in a CAT. Issuers SHOULD include a "kid" header field to communicate to recipients which key is used to sign the token. Recipients MAY reject a token that does not contain a "kid" header field.

Choosing an algorithm: The different algorithms have different properties to recommend them. A full primer on the cryptographic options is out of scope for this document, but issuers should consider these general features when selecting an algorithm:

- HMAC 256/256: This is a symmetric algorithm. It is very fast to both sign and verify with, but the same key is used for both. This means the recipient must possess a copy of the issuer's signing key. Issuers MUST NOT provide a signing key to participants they do not trust to sign tokens.
- PS256: This is an asymmetric algorithm. It is fairly slow to sign with, but very fast to verify with. It has a larger signature size, which increases the overall size of the token.
- ES256: This is an asymmetric algorithm. It is very fast to sign with and has a small signature size, which reduces the overall size of the token. The verification speed is usually slower than PS256.

These guidelines are accurate as of the writing of this document, but implementers will need to benchmark on their own platforms in order to determine which algorithms work best.

4.3.1.2 Terminology

This document reuses terminology from RFC 8392 and RFC 8949 and additionally defines the following terms:

Well-formed: This has the same meaning as defined in RFC 8949 when applied to CBOR data items. When used to describe a CAT, it means that it has a well-formed claim set and a well-formed header.

Valid: This has the same meaning as defined in RFC 8949 when applied to CBOR data items. When used to describe a CAT, it means that the claim set is valid, the header is valid, the syntactic requirements of any present claims are met, and the signature is valid. It does not, however, mean that all the claims are satisfied.

Acceptable: A CAT is acceptable if and only if it is valid and the main claim set is acceptable. A claim set is acceptable if and only if every claim in it is acceptable. A claim is acceptable if what it asserts is either true or not relevant to the recipient. A token that is unacceptable, but still valid, may be subject to the semantics of a "catif" claim (section 4.10.1), if one is present.

Support: For the purposes of this document, the word "support" means not just that a recipient is prepared to accept a valid value, but also that they can enforce it as the claim REQUIRES. A recipient that supports all the REQUIRED components of an OPTIONAL claim supports that claim. Some claims (both MANDATORY and OPTIONAL) have OPTIONAL features. It is not necessary for a recipient to support all OPTIONAL features in order to support a claim.

4.3.1.3 Conventions

This document also uses Concise Data Definition Language (CDDL) to define the CBOR objects. CDDL is defined in RFC 8610. For convenience, it is collected herein in Annex D. The CDDL in this document uses a more permissive license (see section 2) to allow implementations to embed the CDDL directly.

Claims in this document are added to the $$Claims-Set-Claims socket.

Examples in this document also use CBOR Extended Diagnostic Notation (EDN). This is defined in RFC 8949 Section 8 and normatively extended in RFC 8610 Appendix G. CBOR EDN examples are not permissively licensed.

In this document, the format “the type is X” is used to indicate CBOR types. For example, “The type of this claim is array and the elements of the array are text strings.”

4.3.1.4 Scope

This document describes methods for denying and permitting access to content. However, a CAT can be used in conjunction with other mechanisms for permitting or denying access. The ways these mechanisms interact with a CAT are not specified by this document. Rather, this document simply describes whether access is permissible according to the rules herein. The use of a CAT is not mandatory unless a recipient is configured to require it. Likewise, there may be additional mechanisms that deny access, even when an acceptable CAT is present.

When a CAT is required for accessing content, a recipient MUST NOT provide access to that content unless an acceptable CAT is presented.

When a CAT is not required for accessing content, a recipient MUST NOT use the presence or unacceptability of a CAT to deny access to content.

4.3.1.5 Claims

All claims are OPTIONAL for issuers. A claim set consists of zero or more claims, so no claim is required in any given token. Issuers MUST format claims as described herein, but MAY choose to omit any or all claims. Recipients MUST ignore claims they do not understand. (This requirement might be modified by other claims, as described in their definitions. One such claim may be defined in the draft described in Section 3.1.)

Issuers MAY include claims not described herein, but the meaning of those claims is not described by this document. The IANA registry of CWT claims has a list of publicly defined claims. When new claims are necessary, registering them via IANA's process will help avoid conflicts and, when defined properly in a public specification, help implementations interoperate.

An issuer that requires a specific set of claims SHOULD communicate those claims to recipients, but the form of such communication is outside the scope of this document. It could take the form of a CBOR array of key ids or written prose listing the claim names.

Recipients MUST implement the claims that are marked as MANDATORY in this document. A recipient SHOULD communicate which claims they support by listing all the claims they support, including both claims they MUST implement and those they MAY. The exact form of this communication is outside the scope of this document but could take the form of a CBOR array of key ids or written prose in documentation listing the claim names.

A claim set is a CBOR array of zero or more claims, as described in RFC 8392 Section 2 where "CWT Claims Set" is defined. The main claims of a CWT are defined in the base claim set. A claim set is rejected if any claim within it is rejected. A request MUST be rejected if the base claim set in the token is rejected. Composition claims are special claims that contain claim sets themselves and are accepted or rejected based on whether their subclaim sets are accepted or rejected. If a claim causes the token to be invalid, the token is unacceptable. CDDL for a Claim Set is defined by cwt-claims below in section 4.5.

4.3.1.5.1 Caching

One purpose of the CAT is to allow intermediaries to serve cached responses to only authorized clients. To this end, it is important to prevent shared caches from serving cached responses retrieved by way of a token to those without a token. Recipients that process a CAT MUST insert the "private" directive into the Cache-Control field of responses.

Likewise, a recipient that accepts a CAT SHOULD remove it from forwarded requests, if it is practical and reasonable to do so. Sometimes, it cannot reasonably be removed, especially when it is conveyed in the URI, but when it can, it SHOULD be. This reduces the risk that an upstream intermediary also processes the token and uses the "private" directive to prevent the effective caching of the response.

4.3.1.5.2 Watermarking

The DASH-IF Forensic A/B Watermarking specification ETSI TS 104 002 V1.1.1 defines claims for a CBOR Web Token that allow for interoperable watermarking of Adaptive Bit Rate content. The claims defined in that document do not conflict with this document and can be used in tandem.

Implementations using both formats should take note that while this specification permits the omission of the padding (section 4.3.1.1), but the DASH-IF watermarking token does not. Additionally, the CAT relaxes the requirement that maps be presented in a particular order. The watermarking token requires Deterministic Encoding with no exceptions. While the Expiration ("exp") and Issued At ("iat") claims are optional in the CAT, they are required in the watermarking token.

4.4 Token location

Tokens are stored in any of five locations:

- Path-style parameters
- Form-style parameters
- Cookies
- Authorization field (see 4.4.1 below)

- HTTP Field

Path-style parameters generated in the form indicated by RFC 6570 Section 3.2.7, Form-style parameters generated in the form indicated by RFC 6570 Sections 3.2.8 and 3.2.9, and the HTTP Field MUST be supported.

If a CTA-Common-Access-Token header field (or alternate field as described below) is present, a recipient MUST process only that; other locations MUST NOT be scanned for tokens. Otherwise, if more than one token is present and at least one token is acceptable, tokens that are unacceptable MUST be treated as though they were not present. If more than one valid token is present and no token is acceptable, then access to the content will not be permitted by any of the tokens. Nevertheless, claims that affect the handling of unacceptable tokens are processed. Any such claim MUST define a precedence for the claim in the event of multiple unacceptable tokens. Valid tokens are processed in order as they would be found in a left-to-right parsing of the URI, then a left-to-right parsing of the Cookie field.

In this way, any acceptable token provides access to the content. But if there are multiple unacceptable but valid tokens, there is a consistent way of evaluating the claims that can affect the processing of a request with an unacceptable token. In this document, the only such claim is the "catif" claim (which says only the first "catif" claim is to be processed), but other documents may define additional claims.

An implementation MAY limit the number of tokens it is willing to process but MUST process at least six tokens.

The names of the parameters or cookies are subject to configuration and interoperable implementations will need to agree on the name. Likewise, which schemes (DPoP, Bearer, or other schemes) are valid places for a token are subject to configuration. The mechanism by which this occurs is beyond the scope of this document, but in the absence of alternative indications, participants are encouraged to consider the parameter "CAT" as a reasonable default.

If the CAT is conveyed in the authorization field using the DPoP scheme, the recipient MUST treat the request as one for access to a protected resource as described by RFC 9449 Section 7. That means that the "ath" claim in the DPoP proof MUST be provided. If the CAT is conveyed in any other way, the "ath" claim of the DPoP proof is OPTIONAL and a recipient MAY accept a proof that does not provide it. The security considerations described in RFC 9449 Section 7 apply here. Issuers and clients using DPoP should not convey a CAT without using the DPoP authorization scheme if replay of the proof could provide unauthorized access to protected resources.

If the token processing results in the rejection of the request and no "catif" claim specifies specific handling or if a token was expected but not provided, the recipient MAY return a 401 (Unauthorized) status code and the WWW-Authenticate field in order to prompt the client to provide a CAT. The recipient MUST NOT provide access to the resource if token processing results in the rejection of the request.

4.4.1 Authorization field

A CAT can be sent using the Authorization request field using the Bearer and DPoP schemes.

4.4.1.1 DPoP compatibility

Recipients that support DPoP as specified in RFC 9449 MAY use a CAT as an access token as described by that specification. A CAT serving as a DPoP access token will have a "cnf" claim with a "jkt" member, or another member defined for this use. This associates the public key with the token as described in RFC 9449 Section 6.

A CAT presented in an Authorization field using the DPoP scheme is processed along with other tokens that may be present in the URI or Cookie field.

4.4.1.2 Bearer compatibly

CATs can also serve as Bearer tokens as defined by RFC 6750 and observed by RFC 9449 Section 7.2.

A CAT presented in an Authorization field using the Bearer scheme is processed along with other tokens that may be present in the URI or Cookie field. Clients, however, will not know whether the recipient is capable of processing the Bearer token as a CAT or if it simply accepts it as a Bearer token in accordance with the processes in RFC 6750. Furthermore, this document does not define what a recipient should do with a CAT presented as a bearer token. It MAY process it.

4.4.2 HTTP field

A CAT can be sent using either the standard field or a separately configured field. The mechanism by which this field can be configured is out of scope for this document. A separately configured field MUST be treated as though it were the standard field.

The standard field is "CTA-Common-Access-Token". CTA-Common-Access-Token is a Structured Field with a value of type token or string. This item MAY be parameterized, but no parameters have meanings defined in this document. Unknown parameters MUST be ignored. This token or string is the encoded CAT.

CTA-Common-Access-Token is both a request and response field. It is valid on all requests and applies only to the specific request in which it is present. Intermediaries MUST NOT modify or delete this field unless they are configured as a recipient for the CAT. In that case, they SHOULD remove the field before forwarding the request.

It is valid on responses and indicates a new token is being issued pursuant to a "catr" claim. Intermediaries MUST NOT modify or delete this field.

CTA-Common-Access-Token MUST NOT be sent in trailers; any CTA-Common-Access-Token field in the trailers MUST be ignored.

CTA-Common-Access-Token MUST NOT be included in the Connection header field.

CTA-Common-Access-Token MUST NOT be included in a Vary header.

Recipients that process the CTA-Common-Access-Token field MUST add the private directive to the Cache-Control of responses if it is not already present.

CTA-Common-Access-Token MUST NOT be stored as the result of a PUT. Rather, it is intended to be processed to determine authorization for the PUT to be processed.

CTA-Common-Access-Token SHOULD NOT be removed when performing a redirect. It serves as authorization to access the resource, even if that resource must be accessed elsewhere. This behavior is different than when processing a redirect with an Authorization header. Tokens will need to be valid for the target resource in order for the redirection to result in a success, so issuers will need to take this into consideration.

4.5 Claims

Tokens can use any of these claims in order to enforce a policy determined by the issuer. An issuer can use any subset of these claims to define the distribution policy an intermediary serving the request MUST follow.

No claim is required in any given token, but a recipient needs to be able to process all mandatory claims. If a claim is present that the recipient does not understand or support, the recipient has to ignore the claim and continue processing the token as though the claim were not present.

All CBOR MUST be encoded as Deterministically Encoded CBOR as described in RFC 8949 Section 4.2, except that maps need not be encoded in bytewise lexicographic or length-first map key order. Rather, maps SHOULD be encoded with claims the issuer expects to be most likely to fail or most inexpensive to check listed first. Recipients MAY process the claims in any order.

CAT that are not Deterministically Encoded (with the exception of map ordering) MAY be treated as ill-formed and rejected. Recipients SHOULD validate that tokens are Deterministically Encoded.

As required by Deterministic Encoding, the claim values defined herein, with the exception of the "catnip" claim which requires it, MUST NOT be prefixed with a CBOR tag. For example, the "catm" claim could be reasonably described as a homogeneous array and prefixed with tag 41. However, this would add no additional information for the parser since it is already defined as such and it would consume additional space.

Likewise, when the meaning of a value would not change on account of a tag, issuers MUST omit the tag. Recipients MUST reject a claim set containing a claim with a value that is improperly tagged. These requirements apply both to claim values and their subvalues.

Numbers encoded have only their real numeric meaning and MUST be encoded in their shortest accurate form. The first floating point rule in RFC 8949 Section 4.2.2 MUST be used: integral values that fit inside 64 bits MUST be encoded with major types 0 or 1, otherwise the value MUST be encoded as the preferred floating-point type that accurately represents the value. Floating-point NaN values and negative zero floating-point values are not valid values for any claim defined in this document, but they may be valid values for claims defined elsewhere.

The parts of this CDDL that describe structures and claims not defined in this document are informational only and provided for convenience. The parts that describe claims defined herein are normative. Below, informative CDDL is marked as such.

The following CDDL describes CWTs in general and as used in this document:

```
cwt-claims = {
  * $$Claims-Set-Claims
  * label => any
```

```
}

label = int / tstr
label-or-labelset = label / [ + label ]

stringoruri = tstr   ; Interpreted per RFC 8392 Section 2
numericdate = number ; Interpreted per RFC 8392 Section 2
```

Key ID	Claim	Type	Required?	Description
1	iss	text string	MUST	Issuer
3	aud	text string or array	MUST	Audience
4	exp	positive or negative integer or floating-point number	MUST	Expiration
5	nbf	positive or negative integer or floating-point number	MUST	Not Before
7	cti	byte string	MUST	CWT ID
308	catreplay	unsigned integer	MAY	Replay
309	catpor	array	MAY	Probability of Rejection
310	catv	unsigned integer	MUST	Version
311	catnip	array	MUST	Network IP
312	catu	map	MUST	URI
313	catm	array	MUST	Method
314	catalpn	array	MAY	ALPN
315	cath	map	MUST	Header
316	catgeoiso3166	array	MAY	Geographic
317	catgeocoord	array	MAY	Geo Detail
282	geohash	text string or array	MAY	Geohash
318	catgeoalt	array	MAY	Altitude
319	cattpk	byte string	MAY	TLS Public Key

Key ID	Claim	Type	Required?	Description
2	sub	text string	MUST	Subject
6	iat	positive or negative integer or floating-point number	MUST	Issued At
320	catifdata	string or array	MAY	If Data
8	cnf	map	MUST	Confirmation
321	catdpop	map	MUST	DPoP Settings
322	catif	map	MAY	If
323	catr	map	MAY	Renewal

4.6 Core claims

The core claims are claims that form the primary basis for accepting or denying a request.

4.6.1 Issuer (iss) claim

The semantics in RFC 8392 Section 3.1.1 MUST be followed. The recipient MUST reject a claim set that is not part of a token signed with an issuer known to the recipient to be acceptable or that is not signed with a key known to the recipient to be associated with that issuer. The mechanism by which recipients determine which issuers are acceptable and how they receive keys is outside the scope of this document.

Recipients MUST support this claim.

The "iss" claim is defined in RFC 8392 but described here for convenience by the following informational CDDL:

```
$$Claims-Set-Claims //= (iss-label => iss-value)
iss-label = 1
iss-value = stringoruri
```

4.6.2 Audience (aud) claim

The semantics in RFC 8392 Section 3.1.3 MUST be followed. This claim is used to ensure the recipient is an intended recipient of the token. The recipient MUST identify with one of the entities listed as a potential recipient of the claim set.

Recipients MUST support this claim.

The "aud" claim is defined in RFC 8392 but described here for convenience by the following informational CDDL:

```
$$Claims-Set-Claims //= (aud-label => aud-value)
aud-label = 3
aud-value = stringoruri / [ * stringoruri ]
```

4.6.3 Expiration (exp) claim

The semantics in RFC 8392 Section 3.1.4 MUST be followed, except that recipients MUST NOT allow for leeway in time synchronization. A recipient MUST reject a claim set in a request made after this time.

Recipients MUST support this claim.

The "exp" claim is defined in RFC 8392 but described here for convenience by the following informational CDDL:

```
$$Claims-Set-Claims //= (exp-label => exp-value)
exp-label = 4
exp-value = numericdate
```

4.6.4 Not Before (nbf) claim

The semantics in RFC 8392 Section 3.1.5 MUST be followed, except that recipients MUST NOT allow for leeway in time synchronization. A recipient MUST reject a claim set in a request made before this time.

Issuers are advised that setting the "nbf" claim to the time at issuance is potentially dangerous. Because leap seconds can make clocks appear to go backward for an instant, an "nbf" claim may not be redeemable a moment after issuance. Recipients are prohibited from allowing leeway in time synchronization, so issuers must consider any leeway they wish to include in the value of the "nbf" claim directly.

Recipients MUST support this claim.

The "nbf" claim is defined in RFC 8392 but described here for convenience by the following informational CDDL:

```
$$Claims-Set-Claims //= (nbf-label => nbf-value)
nbf-label = 5
nbf-value = numericdate
```

4.6.5 CWT ID (cti) claim

The semantics in RFC 8392 Section 3.1.7 MUST be followed. A recipient can use this field to reduce or prevent the reuse of a token by using it in conjunction with the "catreplay" claim. A recipient that supports token revocation MAY use this claim to reject a claim set in a request with a revoked token. The mechanisms by which tokens can or cannot be revoked are outside the scope of this document.

This value MUST be generated in a way that does not identify the subject of the token (for example, it may be a random byte string). If issuers do not share "cti" claim values and subject identities out-of-band with recipients, this may provide additional privacy for clients in many cases.

The "cti" claim is not a "session identifier" for use in tracking a series of requests that may be related. It is bound to the specific token, so any of the forms of token renewal or reauthorization will have a new and different "cti" claim.

Recipients MUST support this claim.

The "cti" claim is defined in RFC 8392 but described here for convenience by the following informational CDDL:

```
$$Claims-Set-Claims //= (cti-label => cti-value)
cti-label = 7
cti-value = bstr
```

4.6.6 Common Access Token Replay (catreplay) claim

This claim indicates whether token replay is prohibited. The type of this claim is unsigned integer. If the value is zero, replay is permitted, and recipients MUST NOT reject a claim set in a request that bears a token with a "cti" claim that has already been used on account of that reuse. If the value is one, replay is prohibited, and recipients MAY reject a claim set in a request that bears such a token.

If the value is two, the recipient MAY use reuse-detection to deny reuse according to an implementation-defined process. This document does not describe how reuse-detection is done, but a value of 2 here is intended to convey that the issuer would like the claim to be rejected if the recipient determines, according to an implementation-defined process, that the token is being used by multiple clients. This value indicates that the recipient is willing to accept the risk of rare false positive detections resulting in the rejection of a claim from a client that is not using a duplicated token. This may be appropriate if it is easy for clients to obtain new tokens, especially in conjunction with a "catif" claim.

If the value is not one of these, recipient behavior is undefined.

Token replay protection may be available as a best effort, if it is available at all.

The Claim Key 308 is used to identify this claim.

Recipients MAY support this claim.

The "catreplay" claim is identified by the following CDDL:

```
$$Claims-Set-Claims //= (catreplay-label => catreplay-value)
catreplay-label = 308
catreplay-value = $catreplay-value
$catreplay-value /= 0 ; permitted
$catreplay-value /= 1 ; prohibited
$catreplay-value /= 2 ; reuse-detection
$catreplay-value /= uint .gt 2 ; undefined
```

4.6.7 Common Access Token Probability of Rejection (catpor) claim

The "catpor" claim randomly rejects a claim based on a specified probability and allows the recipient to permanently block reuse. The type of this claim is number. The "catpor" value MUST be between 0 and 1, inclusive. A claim with a value greater than 1 or less than 0 is not well-formed and the token MUST be rejected.

The "catpor" value is an array with up to three elements. The first is a number that defines the probability of rejecting the claimset and blocking reuse. A "catpor" claim is rejected with probability equal to that value.

The second member of the array determines which value is used to block reuse. Once a "catpor" claim is a basis for rejection of an otherwise valid claimset, the recipient SHOULD reject "catpor" claims in future tokens with a claim that has a matching value. If the second member is a byte string (major type 2), it is used directly to block future tokens from the same issuer with a "catpor" claim that match this value. If it is an integer (major type 0 or 1), it is a reference to another claim in the same claimset, the

value of which is used to block tokens with a matching value. If another claim is referenced and that claim is not present or does not have a value of type byte string, the token is not well-formed. The issuer is the party that issued the token, which may or may not be documented in an "iss" claim.

The third member of the array, if it is present, is an expiration. This value is optional and provides a lifespan for the identifier. This is a NumericDate that defines the expiration of the "catpor" claim. If a recipient blocks a value based on this "catpor" claim, it MUST NOT be blocked beyond the expiration.

Recipients should ensure an adequate entropy source for this claim, because the security properties depend on truly random rejection. Issuers should also consider that a token that fails for the rejection of the "catpor" claim may also have other rejected claims.

Recipients need not process the "catpor" claim of otherwise unacceptable tokens, so as to save space in recording unacceptable match values. Likewise, entries need to stay on the block list only until the expiration of the token with the "catpor" claim or the expiration of the "catpor" claim itself. Recipients that process requests on multiple systems MAY communicate the list of unacceptable values between the systems on a best-effort basis.

The Claim Key 309 is used to identify this claim.

Recipients MAY support this claim.

The "catpor" claim is described by the following CDDL:

```
$$Claims-Set-Claims //= (catpor-label => catpor-value)
catpor-label = 309
catpor-value = [catpor-probability, catpor-id, ?catpor-exp]
catpor-probability = number
catpor-id = label / bstr
catpor-exp = numericdate
```

4.6.7.1 Example

```
{
      /catpor/ 309: [.00005, h'087ee44f239f7a2e34b3d1649aad8c1d', 1700000000]
}
```

This token indicates that the Probability of Rejection is 0.005%, so one in every 20,000 claimsets will be rejected and blocked. When the claimset is rejected and blocked the byte string `h'087ee44f239f7a2e34b3d1649aad8c1d'` will be recorded on the block list until the listed expiration, which is on November 14, 2023, 22:13:20 GMT (1700000000 Unix time).

If, after that "catpor" claim is blocked, but before the expiration, another token is presented with the same string in the second position (even if it differs in other claims), that "catpor" claim will be rejected. However, this token would not be automatically blocked:

```
{
      /cti/ 7: h'087ee44f239f7a2e34b3d1649aad8c1d'
      /catpor/ 309: [.00005, 7, 1700000000]
}
```

Even though the "cti" claim value matches the blocked byte string and the "catpor" claim references the "cti" claim, it will not match because only tokens with a matching value in the same place are blocked. It will, however, be subject to random rejection and blocking like any "catpor" claim.

After 1700000000 Unix time, the recipient "forgets" the block on that identifier and treats the "catpor" claim as new again, acceptable until it is randomly blocked once more.

4.6.8 Common Access Token Version (catv) claim

This claim specifies the version of the token, to allow future specifications to extend or modify it and maintain backward compatibility. The type is Unsigned Integer and it MUST be set to “1”. A token that does not use this claim can be assumed to be of version “1”. A recipient MUST reject a token with an unsupported version number.

This claim may only be conveyed in a base claim, never in a claim that contains other claims. (No such claims are currently defined, but they may be defined in the future.)

The Claim Key 310 is used to identify this claim.

Recipients MUST support this claim.

The "catv" claim is identified by the following CDDL:

```
$$Claims-Set-Claims //= (catv-label => catv-value)
catv-label = 310
catv-value = 1
```

4.6.9 Common Access Token Network IP (catnip) claim

This claim restricts the usage of the token to a particular IP Address or network. The type of this claim is an array, and the elements of the array are tagged data items described in RFC 9164, or positive integers. Either or both tags 54 and 52 are allowed, as are positive integers, but a network claim with any other data type is invalid. Additionally, a network IP claim that uses the address-with-prefix form is invalid; only the IP address and IP address prefix forms are permissible. An unsigned integer is an Autonomous System Number (ASN) and indicates all networks belonging to that ASN. A recipient MUST reject a claim set with an invalid network claim.

If this claim contains IP addresses, IPv4 address prefixes of length greater than 24, or IPv6 address prefixes of length greater than 56, the value of this claim MUST be encrypted. The value can be encrypted either in a claim with an encrypted value (of which, none are currently defined) or in an encrypted token, although this document does not specify any MANDATORY algorithms for token encryption. Recipients MUST consider a request containing an unencrypted "catnip" claim of greater specificity than above to be invalid. Issuers are encouraged to ensure that they use the most general prefix that is suitable in order to maximize privacy.

The network IP claim describes a list of acceptable networks from which a token may be accepted. A claim from a network that does not match one of the addresses, prefixes, or networks listed MUST be rejected.

As a practical matter, IP addresses are exceptionally poor vehicles for identifying clients across either time or space. Clients are very likely to change IP addresses as they change networks or make requests

from different interfaces or different IP stacks. Any token issuer that includes a network claim can expect that many tokens will be rejected through no fault of the client.

The Claim Key 311 is used to identify this claim.

Recipients MUST support this claim.

The "catnip" claim is identified by the following CDDL:

```
$$Claims-Set-Claims //= (catnip-label => catnip-value)
catnip-label = 311
catnip-value = [ + catnip-object ]


; Autonomous System Number (ASN), IPv6 address or IPv4 address
catnip-object = uint/ ipv6-address-xor-prefix / ipv4-address-xor-prefix


; IP addresses as defined in in [RFC9164] section 5
; excluding the ipv6-address-with-prefix option
ipv6-address-xor-prefix = #6.54(ipv6-address / ipv6-prefix)
ipv4-address-xor-prefix = #6.52(ipv4-address / ipv4-prefix)

ipv6-address = bytes .size 16
ipv4-address = bytes .size 4

ipv6-prefix-length = 0..128
ipv4-prefix-length = 0..32

ipv6-prefix-bytes = bytes .size (uint .le 16)
ipv4-prefix-bytes = bytes .size (uint .le 4)

ipv6-prefix = [ipv6-prefix-length, ipv6-prefix-bytes]
ipv4-prefix = [ipv4-prefix-length, ipv4-prefix-bytes]
```

4.6.10 Common Access Token URI (catu) claim

This claim limits the URI to which the token can provide access. The type is map; the keys are integers, either negative or unsigned; the values are maps. The keys of the map are integers that describe what URI component should be considered. The keys correspond with URI components as defined by RFC 3986:

0: scheme (RFC 3986 Section 3.1)

1: host (RFC 3986 Section 3.2.2)

2: port (RFC 3986 Section 3.2.3)

3: path (RFC 3986 Section 3.3)

4: query (RFC 3986 Section 3.4)

5: parent path

6: filename

7: stem

8: extension

Keys 5 through 6 represent specific subparts of the path. The parent path is the path from the beginning to the penultimate path segment, inclusive, but not including the terminal path separator. The filename is the final path segment, not including any path separators. The stem is the beginning of the filename, up to but not including the last full stop ("."; Unicode 0x2e). If the filename does not contain a full stop, the stem is the full filename. If the filename starts with a full stop, the stem is an empty string. The extension is the portion of the filename after the stem, including the full stop if it is present. If no full stop is present in the filename, the extension is an empty string.

The match value is a map. The keys are integers that describe the match components:

0: Exact text match. The type for the value is text string.
1: Prefix Match. The type for the value is text string.
2: Suffix Match. The type for the value is text string.
3: Contains Match. The type for the value is text string.
4: Regular Expression Match. The type for the value is an array of text strings. The first element of the array is a regular expression, and subsequent elements are fields for matching. Fields for matching may be null.
-1: SHA-256 match. The type for the value is byte string. Support for this match type is OPTIONAL.
-2: SHA-512/256 match. The type for the value is byte string. Support for this match type is OPTIONAL.

Future specifications may define additional values for this map, but they will not define values at or below -25. Values at or below -25 are reserved for private match types.

A recipient MUST reject a claim set in a request for a URI that does not match the components in the "catu" claim. Additionally, it MUST reject a claim that contains match types that it does not support.

Before performing the match, the recipient MUST remove the token from the URI if it is present. This removal is only for the purposes of matching. If the token is terminated by anything other than a sub-delimiter as defined by RFC 3986 Section 2.2, everything from the reserved character as defined by RFC 3986 Section 2.2 that precedes the path- or form-style parameter name to the last character of the token is removed. Otherwise, everything from the beginning of the path- or form-style parameter name to and including the sub-delimiter that terminates it is removed.

Then, normalize the URI according to RFC 9110 Section 4.2.3 and then RFC 3986 Sections 6.2.2 and 6.2.3.

Once the token is removed and the URI is normalized, each member of the map is compared with the corresponding component of the URI using the indicated matching type. If any value does not match, the claim set MUST be rejected.

Exact Text Match: Exact text matching is performed by performing a case-sensitive character-by-character comparison of the component and the text to match against. They match if they are exactly the same.

Prefix Match: Prefix matching is the same as an exact text matching, except that they match if the prefix string matches the beginning of the component. A prefix need not end exactly at a path segment or DNS segment boundary; issuers that require that will need to include terminal delimiters in the prefix.

Suffix Match: Suffix matching is the same as prefix matching, except that they match if the suffix matches the end of the component.

Contains Match: Contains matching is the same as prefix matching, except that they match if the string matches anywhere inside the component.

Regular Expression Match: The text to match against is an Extended Regular Expression as defined in IEEE Std 1003.1-2017 Section 9.4. The expression is not anchored to either the start or the end of its input, although anchoring symbols may be used to accomplish this. The expression is case-sensitive. This matches if the Regular Expression accepts the component.

A regular expression match can contain additional strings or null values after the regular expression in the match array. These match, in order, groups defined in the regular expression. The regular expression match only matches if each additional string is an exact text match for the respective group in the regular expression. It always matches a null value, which indicates that the group for the corresponding entry need not be compared to anything. The second element (immediately after the regular expression string) corresponds with the first group in the regular expression, the third element corresponds with the second, and so on.

If more strings are present after the regular expression than the regular expression has groups, some of those strings will not match against groups and result in the rejection of the claim. This is true even if the last values of the array are null, because null matches any value, not the absence of a group. Not all groups in the regular expression need to have array values against which to match.

The maximum length of this claim is effectively controlled by the maximum length of the token, but recipients MUST implement resource limits on the computation consumed by this claim. The length of the claim is not a useful proxy for complexity, so simple analysis of the claim value is of limited value.

SHA-256 Match: The component is hashed using SHA-256 as defined in FIPS-180-4. The resultant bytes are compared to the byte string to match against. They match if they are exactly the same.

SHA-512/256 Match: The component is hashed using SHA-512/256 as defined in FIPS-180-4. The resultant bytes are compared to the byte string to match against. They match if they are exactly the same.

The Claim Key 312 is used to identify this claim.

Recipients MUST support this claim.

The catu claim is described by the following CDDL:

```
$$Claims-Set-Claims //= (catu-label => catu-value)
catu-label = 312
catu-value = {
      * uripart => match
}

uripart = &(
      scheme: 0,
      host: 1,
      port: 2,
      path: 3,
      query: 4,
      parent-path: 5,
      filename: 6,
      stem: 7,
      extension: 8,
)


match = {
      ? exact-match ^ => tstr,
      ? prefix-match ^ => tstr,
      ? suffix-match ^ => tstr,
      ? contains-match ^ => tstr,
      ? regex-match ^ => [ tstr, * tstr ],
      ? sha256-match ^ => bstr,
      ? sha512-256-match ^ => bstr
}

exact-match = 0
prefix-match = 1
suffix-match = 2
contains-match = 3
regex-match = 4
sha256-match = -1
sha512-256-match = -2
```

4.6.10.1 Examples

Exact Match

```
{
      /catu/ 312: {
            /scheme/ 0: {
                  /exact/ 0: "https"
            }
      }
}
```

This token matches a URI with a scheme exactly equal to "https". It will match a URI presented as "HTTPS://example.com/" because the scheme will be converted to lowercase during normalization.

Multiple Match

```
{
	/catu/ 312: {
		/scheme/ 0: {
			/exact/ 0: "https"
		},
		/path/ 3: {
			/prefix/ 1: "/content"
		},
		/extension/ 8: {
			/exact/ 0: ".m3u8"
		}
	}
}
```

This token matches a URI with all of the following traits:

- A scheme exactly equal to "https";
- A path that starts with "/content"; and
- An extension that is exactly ".m3u8".

This will match a URI that looks like "https://example.com/content/path/file.m3u8". It does not match a URI that looks like "https://example.com/CONTENT/path/file.m3u8". The path is case sensitive and the case is not modified in the normalization step.

Extension Match

```
{
	/catu/ 312: {
		/filename/ 6: {
			/suffix/ 2: ".tar.gz"
		}
	}
}
```

This token matches URIs that have a filename that end in ".tar.gz". For example: "https://example.com/file.txt.tar.gz".

```
/ NOTE: This next example is technically well-formed, but logically useless. /
{
	/catu/ 312: {
		/extension/ 8: {
			/exact/ 0: ".tar.gz"
		}
	}
}
```

This token matches no possible URIs. It is well-formed, but it is impossible for any URI to satisfy the requirement it imposes. The extension starts with the last full stop, if it is present, so it can by definition never contain two full stops.

```
{
	/catu/ 312: {
```

```
            /extension/ 8: {
                  /exact/ 0: ""
            }
      }
}
```

This token matches URIs that have no extension. For example: "https://example.com/file".

```
{
      /catu/ 312: {
            /extension/ 8: {
                  /exact/ 0: "."
            }
      }
}
```

This token matches URIs that have a full stop at the end of the filename. For example: "https://example.com/file.". It does not match a URI that has no full stop at all.

Parent Path Match

```
{
      /catu/ 312: {
            /parent path/ 5: {
                  /suffix/ 2: "content"
            }
      }
}
```

This token matches URIs that end in "content" in the penultimate segment. For example: "https://example.com/files/secret-content/secret.txt".

```
{
      /catu/ 312: {
            /parent path/ 5: {
                  /suffix/ 2: "/content"
            }
      }
}
```

This token matches URIs that end in "/content" in the penultimate segment. It does not match the example above that ends with "/secret-content". Instead, it matches these URIs:

- https://example.com/files/content/file.txt
- https://example.com/content/file.txt

SHA-256 Match

```
{
      /catu/ 312: {
            /parent path/ 5: {
                  /SHA-256/ -1:
      h'14608dd0f26a05afce0b80443c6d3643bc84771469748314808d64381b99898c'
            }
```

```
	}
}
```

and

```
{
	/catu/ 312: {
		/parent path/ 5: {
			/SHA-512 256/ -2:
	h'1444a2e87a280083fdb4b8a3e32a8ea391551c1d071fba1281d7b2a4bc74ef75'
		}
	}
}
```

These tokens match URIs that have a parent path that match the specified hash. In this case, it matches: "https://example.com/highly/nested/file/structure/that/extends/on/and/takes/quite/a/bit/of/space/and/might/consume/more/space/in/the/token/than/is/otherwise/preferable/file.txt".

Normalized Host Match

```
{
	/catu/ 312: {
		/host/ 1: {
			/suffix/ 2: ".example.net"
		}
	}
}
```

This token matches requests for "https://null.example.net/content.ts" as well as "https://NULL.eXaMpLe.nEt/content.ts" because the URI will be normalized prior to matching.

It also matches "https://great.example.net/content.ts" and "https://sub.domain.great.example.net/content.ts" because ".example.net" is a suffix match.

Port Match

```
{
	/catu/ 312: {
		/port/ 2: {
			/exact/ 0: "8080"
		}
	}
}
```

This token matches any request for port 8080. It matches both of these URIs:

- http://example.com:8080/content.ts
- https://example.net:8080/content.ts

Normalized Port Match for Default Ports

The behavior for default ports may be slightly surprising, however. Consider:

```
{
	/catu/ 312: {
		/port/ 2: {
			/exact/ 0: ""
		}
	}
}
```

This token matches any request that matches the default port for the scheme being used. For example, it matches:

- http://example.com/
- http://example.com:/
- http://example.com:80/
- https://example.com/
- https://example.com:/
- https://example.com:443/

It is tempting, but wrong, to create a token that looks like this:

```
/ NOTE: This next example is technically well-formed, but logically useless. /
{
	/catu/ 312: {
		/port/ 2: {
			/exact/ 0: "443"
		}
	}
}
```

This token does NOT match "https://example.com:443", because the port part will be removed in normalization because port 443 is the default port for https. The correct way to accomplish this is to match on two parts:

```
{
	/catu/ 312: {
		/scheme/ 1: {
			/exact/ 0: "https"
		},
		/port/ 2: {
			/exact/ 0: ""
		}
	}
}
```

Filename Match

```
{
	/catu/ 312: {
		/filename/ 6: {
			/exact/ 0: "content.tar.gz"
		}
	}
}
```

This token will match all of these URIs:

- https://example.com/content.tar.gz
- https://example.com/path/to/content.tar.gz?foo=query&bar=yes
- https://example.com/%63ontent.tar.gz

It does not match any of these URIs:

- https://example.com/a/content
- https://example.com/content.xml
- https://example.com/a/CONTENT (the path comparison is case-sensitive)
- https://example.com/a-content
- https://example.com/content-tar-gz
- https://example.com/content/subcontent

Stem Match

```
{
	/catu/ 312: {
		/stem/ 7: {
			/exact/ 0: "content"
		}
	}
}
```

This token will match all of these URIs:

- https://example.com/a/content
- https://example.com/content.xml
- https://example.com/content.tar.gz
- https://example.com/path/to/content.tar.gz?foo=query&bar=yes
- https://example.com/%63ontent.tar.gz

It does not match any of these URIs:

- https://example.com/a/CONTENT (the path comparison is case-sensitive)
- https://example.com/a-content
- https://example.com/content-tar-gz
- https://example.com/content/subcontent

4.6.11 Common Access Token Methods (catm) claim

This claim limits the HTTP methods by which the token can be used. The type is array, and the members of the array are text strings. A recipient MUST reject a claim set in a request that uses a method that is not listed in this claim. If the "catm" claim is not present, any method is acceptable. Methods and members of the "catm" claim are compared case-sensitively. Issuers SHOULD NOT put duplicate entries in "catm" claims.

Recipients MAY have a limit on the number of members of a "catm" claim they are willing to process. If the recipient receives a request with a "catm" claim larger than it is willing to process, it MUST consider the token to be invalid. Recipients MUST process "catm" claims with 50 or fewer elements.

The Claim Key 313 is used to identify this claim.

Recipients MUST support this claim.

The "catm" claim is described by the following CDDL:

```
$$Claims-Set-Claims //= (catm-label => catm-value)
catm-label = 313
catm-value = [ + tstr ]
```

4.6.12 Common Access Token ALPN (catalpn) claim

This claim limits the TLS ALPNs over which the token can be used. The type is array, and the members of the array are byte strings. A recipient MUST reject a claim set in a request that is made over a TLS connection identified by an ALPN that is not listed in this claim. If the "catalpn" claim is not present, any ALPN is acceptable. ALPNs and members of the "catalpn" claim are compared byte-wise and must match exactly. Issuers SHOULD NOT put duplicate entries in "catalpn" claims.

Recipients MAY have a limit on the number of members of a "catalpn" claim they are willing to process. If recipient receives a request with a "catalpn" claim larger than it is willing to process, it MUST consider the token to be invalid. Recipients MUST process "catalpn" claims with 50 or fewer elements.

If a recipient does not determine the ALPN over which a given request is made, the request MUST be rejected. A "catalpn" claim in a request that is not made over a TLS connection MUST be rejected.

The Claim Key 314 is used to identify this claim.

Recipients MAY support this claim.

The "catalpn" claim is described by the following CDDL:

```
$$Claims-Set-Claims //= (catalpn-label => catalpn-value)
catalpn-label = 314
catalpn-value = [ + bstr ]
```

4.6.13 Common Access Token Header (cath) claim

This claim limits the request headers that are acceptable for this token. The type of this claim is map; the keys of this claim are text strings that match HTTP header names, and the values of this claim are arrays with the same meaning as the arrays in the map that is the value of the "catu" claim (section 4.6.11). A recipient MUST reject a "cath" claim for a request that does not satisfy every member of the "cath" claim.

A "cath" claim for a given header never matches if that header is missing, even if the header is present but empty.

A member of the "cath" claim is satisfied if and only if a header field of that name is present and the value of that field matches the value of the member of the claim. Header field names are compared to keys of the "cath" claim case-insensitively. Value matching is performed as described in the "catu" claim.

An issuer MUST NOT issue a token that contains a "cath" claim with two keys that match when compared case-insensitively. A recipient MAY consider a request with a token that contains a "cath" claim with two keys that match when compared case-insensitively to be invalid.

If a header field is represented by more than one field line, the field lines MUST be combined into the field value as defined by RFC 9110 Section 5.2 before it is compared, all at once, to the regular expression.

The Claim Key 315 is used to identify this claim.

Recipients MUST support this claim.

The "cath" claim is described by the following CDDL:

```
$$Claims-Set-Claims //= (cath-label => cath-value)
cath-label = 315
cath-value = {
      * text => match ; match is defined in [4.6.10]
}
```

4.6.13.1 Examples

```
{
      /cath/ 315: {
            "User-Agent": { /contains/ 3: "Mozilla" },
            "My-Example-Header": { /exact/ 0: "CustomValue" }
      }
}
```

This claim set accepts any request that is accompanied by a User-Agent header that contains the string "Mozilla" anywhere within and has a My-Example-Header with the exact value "CustomValue". Of course, because the client can freely set these headers to any values they wish, this does not provide a meaningful restriction.

4.6.14 Common Access Token Geographic ISO3166 (catgeoiso3166) claim

This claim limits access to clients accessing the recipient from a given country and/or region. The type of this claim is array, and the elements of the array are text strings. The text strings are ISO 3166 country and region codes: a country code first, then optionally a dash followed by a region code for that country. A token with a "catgeoiso3166" claim value with an invalid country code or region code MUST be considered invalid. A "catgeoiso3166" claim in a request from a location not listed in the "catgeoiso3166" claim MUST be rejected.

The method by which recipients determine the country from which a request originates is beyond the scope of this document. Likewise, the meaning of the term “originates” is determined by the recipient and beyond the scope of this document.

The Claim Key 316 is used to identify this claim.

Recipients MAY support this claim.

The "catgeoiso3166" claim is described by the following CDDL:

```
$$Claims-Set-Claims //= (catgeoiso3166-label => catgeoiso3166-value)
catgeoiso3166-label = 316
catgeoiso3166-value = [ + tstr ]
```

4.6.15 Common Access Token Geographic Coordinate (catgeocoord) claim

This claim limits access to clients accessing the recipient from a given geographical location. The type of this claim is array, and the elements of the array are arrays of length three that represent a location and distance. This claim is always an array of location arrays, even if only one location is present. While most claims use a single element instead of an array when a claim only has one part, this claim always requires the outside array. This prevents a parser from having to parse the contents of the first array before determining whether it's an array of locations or simply a location.

The first element is a number that represents the latitude, in decimal degrees. Positive values indicate locations north of the equator and negative south.

The second element is a number that represents the longitude, in decimal degrees. Positive values indicate locations east of the prime meridian and negative west.

The third element is an unsigned integer that represents the radius in meters the claim requires the client to be within.

A recipient MUST reject a claim set in a request from a client that is not within at least one of the zones in the "catgeocoord" claim.

A "catgeocoord" claim that is not tagged with a Coordinate Reference System (CRS) Wrapper (tag 279) MUST be interpreted in the WGS84 CRS as defined in DMA TR 8350.2. Recipients MUST support WGS84, but MAY support other CRSs. A recipient MUST reject "catgeocoord" claims that indicate coordinate systems it does not support. Issuers MUST NOT use CRS Wrapper tags to explicitly specify a CRS that it is already specified by the context. So, for example, an issuer MUST NOT use a CRS Wrapper tag around a "catgeocoord" claim to specify a WGS84 CRS because it is implied by the context, but MAY use a CRS Wrapper tag to specify a different CRS. Recipients MAY reject claims with unnecessary CRS Wrapper tags.

Recipients MUST consider a request that contains a "catgeocoord" claim that is not conveyed within an encrypted claim to be invalid. Issuers MUST convey "catgeocoord" claims within an encrypted claim. A "catgeocoord" claim that is conveyed in a token that is encrypted (and not simply signed or MACed) is also encrypted, although this document does not define any MANDATORY algorithms for token encryption.

The Claim Key 317 is used to identify this claim.

Recipients MAY support this claim.

The "catgeocoord" claim is described by the following CDDL:

```
$$Claims-Set-Claims //= (catgeocoord-label => catgeocoord-value)
catgeocoord-label = 317
catgeocoord-value = [ + location ] / #6.279([crs: any, [ + location ]])
location = [ latitude: number, longitude: number, radius: number ]
     / #6.279([crs: any, [latitude: number, longitude: number, radius: number ]])
; Tag 279 is the Coordinate Reference System Wrapper defined in [GEOHASH]
```

4.6.16 Geohash (geohash) claim

The semantics in CTA-5009-A Section 14 MUST be followed. A "geohash" claim MUST be rejected if the request does not come from a region described by a Geohash in the claim.

A "geohash" claim that is not tagged with a CRS Wrapper (tag 279) MUST be interpreted in the WGS84 CRS as defined in DMA TR 8350.2. Recipients MUST support WGS84, but MAY support other CRSs. A recipient MUST reject "geohash" claims that indicate coordinate systems it does not support. Issuers MUST NOT use CRS Wrapper tags to explicitly specify a CRS that it is already specified by the context. So, for example, an issuer MUST NOT use a CRS Wrapper tag around a "geohash" claim to specify a WGS84 CRS because it is implied by the context, but MAY use a CRS Wrapper tag to specify a different CRS. Recipients MAY reject claims with unnecessary CRS Wrapper tags.

A "geohash" claim MUST be conveyed in an encrypted claim if it contains a Geohash longer than four characters in length. A recipient MUST reject a claim set that includes a "geohash" claim with a geohash that is longer than four characters in length if it is not conveyed in an encrypted claim.

This length is not a guarantee of privacy, but rather a bound beyond which privacy is very likely to be compromised by disclosure of this value. Even a four-character Geohash can identify people and places in sufficiently sparse population centers. The Privacy Considerations laid out in CTA-5009 apply here as well. Issuers MUST convey any "geohash" claims that would identify people, households, or specific places in an encrypted claim. Recipients MUST reject "geohash" claims that are not conveyed in an encrypted claim if the recipient determines that the claim identifies people, households, or specific places.

A "geohash" claim that is conveyed in a token that is encrypted (and not simply signed or MACed) is also encrypted, although this document does not define any MANDATORY algorithms for token encryption.

Recipients MAY support this claim.

This claim is defined in CTA-5009, but is described here for convenience as it was at the time of publication:

```
$$Claims-Set-Claims //= (geohash-label => geohash-value)
geohash-label = 282
geohash-value = untagged-geohash-value
      / #6.279([crsw-system, untagged-geohash-value])
untagged-geohash-value = geohash-string
      / [ * geohash-string ]
geohash-string = tstr
      / #6.279([crsw-system, tstr])

crsw-system = any ; interpreted as tag 104 when untagged
```

4.6.17 Common Access Token Altitude (catgeoalt) claim

This claim limits access to only clients in a specific altitude range. A "catgeoalt" claim MUST be rejected if the request does not come from an altitude within the range described in the claim. The type of this claim is array, and it contains exactly two numbers: an altitude in meters and a deviation above and below that altitude, also in meters. These two numbers describe an altitude range that begins at the altitude minus the deviation and ends at the altitude plus the deviation.

Implementation will need to ensure that the deviation that they allow is sufficient to allow for inaccuracies in the source altitude data, whether measured on-device or by other mechanisms. Not allowing enough variation to include all intended altitudes may result in claim rejections that are unintended.

The altitude is measured in meters above the vertical datum. Negative altitudes are permitted and describe locations below the vertical datum. If the "catgeoalt" claim is not tagged with a CRS Wrapper, it MUST be interpreted in the WGS84 CRS as defined in DMA TR 8350.2. Recipients MUST support WGS84 but MAY support other CRSs. A recipient MUST reject "catgeoalt" claims that indicate coordinate systems it does not support. Issuers MUST NOT use CRS Wrapper tags to explicitly specify a WGS84 CRS because it is implied by the context, but MAY use a CRS Wrapper tag to specify a different CRS. Recipients MAY reject claims with unnecessary CRS Wrapper tags.

The Claim Key 318 is used to identify this claim.

Recipients MAY support this claim.

The "catgeoalt" claim is described by the following CDDL:

```
$$Claims-Set-Claims //= (catgeoalt-label => catgeoalt-value)
catgeoalt-label = 318
catgeoalt-value = altitude-spec / #6.279([crs: any, altitude-spec])
altitude-spec = [ altitude: number, deviation: number ]
```

4.6.18 Common Access Token TLS Public Key (cattpk) claim

This claim limits access to only clients that present a certificate chain that contains a certificate that matches. The type of this claim is ByteString and it contains a Subject Public Key Info block as defined in RFC 5280 Section 4.1.2.7, DER encoded. A recipient MUST reject a "cattpk" claim in a request that is presented on a TLS or QUIC connection that does not contain a certificate in the client certificate chain that matches the value of the "cattpk" claim.

The Claim Key 319 is used to identify this claim.

Recipients MAY support this claim.

The "cattpk" claim is described by the following CDDL:

```
$$Claims-Set-Claims //= (cattpk-label => cattpk-value)
cattpk-label = 319
cattpk-value = bstr
```

4.7 Informational claims

4.7.1 Subject (sub) claim

The semantics in RFC 8392 Section 3.1.2 MUST be followed. If this claim contains information that could identify a person, household, or similar entity with privacy interests, its value MUST be encrypted.

Recipients MUST support this claim.

The "sub" claim is defined in RFC 8392, but described here for convenience by the following informational CDDL:

```
$$Claims-Set-Claims //= (sub-label => sub-value)
sub-label = 2
sub-value = stringoruri
```

4.7.2 Issued At (iat) claim

The semantics in RFC 8392 Section 3.1.6 MUST be followed. This claim is only informational and not usually a basis for rejecting a claimset. This claim may inform the processing of other claims, however. For example, it may be used when determining how to handle the processing of a request that is the result of a "catif" claim redirection, which is out of the scope of this document.

Recipients MUST support this claim.

The "iat" claim is defined in RFC 8392 but described here for convenience by the following informational CDDL:

```
$$Claims-Set-Claims //= (iat-label => iat-value)
iat-label = 6
iat-value = numericdate
```

4.7.3 Common Access Token If Data (catifdata) claim

The semantics of this claim are entirely application-specific. The type of a "catifdata" claim is text string or array. This claim is set by tokens created as a consequence of the catif claim in some circumstances. It MUST NOT be used by issuers except in the context of a "catif" claim.

Recipients MAY support this claim. This information may determine recipient behavior in ways that are not defined in this document.

The Claim Key 320 is used to identify this claim.

The "catifdata" claim is described by the following CDDL:

```
$$Claims-Set-Claims //= (catifdata-label => catifdata-value)
catifdata-label = 320
catifdata-value = string / [ * string ]
```

Although a received "catifdata" claim value may contain only a string or array of strings, a "catifdata" claim embedded in a "catif" claim may contain other values, like maps and alternatives, as defined in the "catif" claim.

4.8 DPoP claims

4.8.1 Confirmation (cnf) claim

The semantics in RFC 8747 Section 3.1 MUST be followed. However, this document defines an additional confirmation value: jkt.

The jkt confirmation method has key id 323 with a value of type byte string. This value is the JWK SHA-256 Thumbprint, as defined by RFC 7638, of a DPoP public key. The length of the byte string will be 32, since that is the length of a SHA-256 hash.

The jkt confirmation method is not the same as the ckt confirmation method defined in RFC 9679. The ckt confirmation method is computed on the COSE Key Thumbprint. A jkt confirmation method may be preferable when the Proof of Possession is provided in the form of a JWT.

A recipient MUST reject a "cnf" claim in a CAT that has a "jkt" member that is not associated with a verifiable DPoP proof provided in the way described by RFC 9449. A recipient MAY reject a "cnf" claim in a request that has another confirmation method that is not satisfied, but this document does not describe those methods or their indications for use.

A recipient receiving a CAT MUST process the DPoP claim as a request for Protected Resource Access as defined by RFC 9449 Section 7.

Recipients MUST support this claim. Furthermore, they MUST support the jkt confirmation method. Additional confirmation methods, including the ckt method, MAY be supported.

The "cnf" claim is defined in RFC 8747 but is described here for convenience by the following informational CDDL:

```
$$Claims-Set-Claims //= (cnf-label => cnf-value)
cnf-label = 8
cnf-value = {
  ? cnf-jkt-label ^ => cnf-jkt-value
  * label => any ; other values described in the IANA registry
}

cnf-jkt-label = 323
cnf-jkt-value = bstr .size 32
```

4.8.2 Common Access Token DPoP Settings (catdpop) claim

This claim defines processing semantics for an accompanying DPoP proof. The type of this claim is map. The keys of the map are ints that determine which setting is being set. The value types depend on the setting:

- -1 (crit): array of int
- 0 (window): uint
- 1 (jti): uint

Recipients MUST ignore a setting they do not understand, except insofar as it may affect the processing of a critical setting value.

Other than the reserved -1 value, no future specifications will register a negative DPoP setting value. These negative setting values may be used by private agreement.

The Claim Key 321 is used to identify this claim.

Recipients MUST support this claim.

The "catdpop" claim is described by the following CDDL:

```
$$Claims-Set-Claims //= (catdpop-label => catdpop-value)
catdpop-label = 321
catdpop-value = {
  ? -1 ^ => [ *int ] ; crit
  ?  0 ^ => uint     ; window
  ?  1 ^ => uint     ; jti
  * int  => any
}
```

4.8.2.1 Critical setting

A DPoP setting key of -1 is a list of critical settings. The type of this is array of integers. If a recipient does not understand a DPoP setting with a key listed in this field, the "catdpop" claim must be rejected. This array MUST NOT contain the values -1, 0, or 1, as these settings MUST always be understood.

4.8.2.2 Window setting

A DPoP setting of key 0 is the acceptable time window of a DPoP proof as defined by RFC 9449 Section 4. The type of this claim is unsigned integer. The value defines the acceptable time window, in integer seconds, for accepting a DPoP proof as defined by RFC 9449 Section 4.3.

A recipient MUST use this value to define the window if it is provided. If it is not provided, a recipient MAY determine their own acceptable window in an unspecified way.

The DPoP window setting MUST NOT be the basis for rejecting the "catdpop" claim. However, as a result of this claim, a DPoP proof might be considered invalid and thereby cause the rejection of a "cnf" claim.

4.8.2.3 jti setting

A DPoP setting of key 1 defines processing semantics for the "jti" claim of a DPoP proof as defined by RFC 9449 Section 4. The type of this claim is unsigned integer. If the value is:

> 0: The "jti" claim in the DPoP proof MUST be ignored. This means that a well-formed value of a "jti" claim in the DPoP proof, if present, MUST not be used as a basis for the rejection of a claim set. When this claim is zero, the "jti" claim does not protect the proof against replay attacks.
>
> 1: The "jti" claim in the DPoP proof MAY be honored. If the recipient has a list of previously used "jti" claims, it MAY consider the DPoP proof to be invalid if the presented "jti" claim has already been used. Recipients MAY use best effort heuristics to determine if a given "jti" has been used but MUST ensure that they do not incorrectly reject a DPoP proof that does not have a "jti" claim that has been used within the window for which it was valid (see the window setting in 4.8.2.2).

If the "jti" setting is missing or set to a value the recipient does not understand, recipients MUST treat it as though it were present and set to zero.

The "jti" setting MUST NOT be the basis for rejecting the "catdpop" claim. However, as a result of this claim, a DPoP proof might be considered invalid and thereby cause the rejection of a "cnf" claim.

4.9 Request claims

4.9.1 Common Access Token If (catif) claim

This claim allows a token issuer to define explicitly how a request with no acceptable token should be handled. The type of this claim is a map; the keys of the map are either claim keys or arrays of claim keys and the values are arrays; the arrays have up to three elements: the status code to return, a map with headers and values (the header map), and a TextString that identifies a key to sign the resultant URI with (the key id). The status code is an unsigned integer, the keys of the map are TextStrings and values of the map are either TextStrings or Arrays. If a value is an array, it is an array of TextStrings and Maps, which will be processed and concatenated as described below.

This claim affects the processing of requests with valid but unacceptable tokens. In the event that there are multiple such tokens, the first token (see Section 4.4) with a "catif" claim that can affect the response MUST do so. If access to content is denied for reasons other than an unacceptable token, the recipient MAY choose to ignore the token entirely, including its "catif" claim.

If this token is unacceptable, if one of the keys of the "catif" claim matches a key (either by matching the map key or by matching a claim key that is a member of the map key) that was a reason for that rejection, the response MUST use the status code indicated for one of those keys and include the indicated headers for that key. For each element of the header map, if the response does not already contain a header field of that name and if the recipient is not configured explicitly to not send a header field of that name, the recipient MUST add a header field of that name with the indicated value to the response. If the response does already contain a header field of that name or if the recipient is explicitly configured not to send a header field of that name, the recipient MUST NOT update the value of that header unless it is configured explicitly to do so. The mechanism by which this might be configured is out of scope for this document.

If the value of the header map is an array, process it by first converting any Maps it contains into a TextString that is a fully encoded CAT, presented in base64url as usual. To convert the Map to a CAT, treat the map as a claim set. For each of the following claims, if the value of the claim is null (which is not usually a valid claim value), replace the null value with the appropriate information:

- iss: A TextString that the recipient identifies with as an issuer.
- iat: A value that identifies the time at which the CAT was issued.

If the map contains a "catifdata" claim, process the value of that claim as though it were a "catif" claim action header value, except that no maps are converted to CATs and strings are not concatenated. That is, use a string value directly and convert arrays by processing the values, converting maps to empty strings and alternatives to that which they represent.

All claim sets MUST be processed in this way, including claim sets nested within other claim sets, with the exception of claim sets entirely contained within a claim with an encrypted value (of which none are currently defined). Claim sets within such a claim MUST NOT be processed in this way, since they are encrypted and cannot be modified without an appropriate encryption key. Indeed, without an appropriate decryption key, it may not even be possible to identify them.

All other claims are used as they are presented. Then, the claims are signed or MACed using the key identified in the third member of the array. If no such key is identified or if the recipient cannot or does not choose to sign or MAC the CAT, an empty string is used instead of an encoded CAT.

Once maps are converted to CATs, process it again by converting any integers into strings. The integer substitutions are defined here:

- 0: This integer is converted to the URI that the client originally requested, encoded in base64url, with no padding (RFC 4648 Section 5 and 3.2). The type of this tag is unsigned integer.
- 1: The same as above for 0, but the CAT being processed is removed from the URI as defined in the "catu" claim (section 4.6.10) first.
- 2: This integer is converted to a CBOR list of key labels, which are the labels of the keys that were rejected, encoded in base64url, with no padding (RFC 4648 Sections 5 and 3.2). Recipients MUST include at least one key label that corresponds with a rejected claim if the claimset was rejected

on account of a claim, but MAY include multiple labels if multiple claims could form the basis for rejection. Issuers should be aware of the fact that the client will receive this information and providing detailed rejection information is not usually a good security practice. However, in many cases, the client will already have enough information to validate the claim set and this does not disclose additional information. If the rejection is based on information the client does not have (especially in the case of encrypted claims), however, this can leak information to them.

- Any negative integer: Reserved for private use.

Future integers and their replacements may be defined in later specifications, but no future specifications will define negative integer replacers. Unknown integers must be replaced with empty strings. Recipients MUST process tokens that contain fewer than 50 elements in all Arrays within the header map, added together.

If a token is rejected on account of a claim that contains other claim sets because of the rejection of one of those claim sets, the recipient MUST process a "catif" claim within one of those claim sets if one is present.

If more than one member of a "catif" claim would be processed, either because multiple members of a single "catif" claim are a basis for rejecting a token or because multiple "catif" claims could be processed because of unsatisfied composition claims being a basis for rejecting a token, the recipient MUST process exactly one of them, but MAY choose among any of them.

A "catif" claim MUST NOT be processed for an invalid token. Likewise, it MUST NOT be processed for a token not signed with a key the recipient can validate.

The Claim Key 322 is used to identify this claim.

Recipients MAY support this claim, and if they do so MAY choose not to sign or MAC CATs embedded in header field values. Recipients MAY either prioritize their own configurations or the "catif" claim.

The "catif" claim is described by the following CDDL:

```
$$Claims-Set-Claims //= (catif-label => catif-value)
catif-label = 322
catif-value = { * label-or-labelset => catif-action }
catif-action = [
  http-status: uint, ; RFC9110 Section 15
  ? headers: catif-headers,
  ? kid: tstr, ; RFC9052 Section 7.1
]
catif-headers = {
  * catif-header-name => catif-header-value
}
catif-header-name = tstr
catif-header-value = tstr / [ + catif-header-value-element ]
catif-header-value-element = tstr / int / cwt-nullable-claims
cwt-nullable-claims = { * label => any }
```

4.9.1.1 Example

```
{
  / exp / 4: / 2023-11-14 22:13:20 / 1700000000
```

```
  / catif / 322: {
    / exp / 4: [
      307,
      { "Location": "http://auth.example.net/" }
    ]
  }
}
```

If this token is presented after its expiration on November 14th, 2023, a recipient that processes the "catif" claim may return a response that looks like this:

```
HTTP/1.1 307

Location: http://auth.example.net/
```

4.9.1.2 Example 2

```
{
  / exp / 4: / 2023-11-14 22:13:20 / 1700000000,
  / catif / 322: {
    / exp / 4: [
      307,
      {
        "Location": ["https://auth.example.net/?CAT=", {
          / iss / 1: null,
          / iat / 6: null,
          / catu / 312: { 1: [0, "auth.example.net"], 2: [0, "/"] },
          / catifdata / 320: [1]
        }]
      },
      "abc123"
    ]
  }
}
```

If this token is presented after its expiration, a recipient may return a response that looks like:

```
HTTP/1.1                                                                    307

Location:  https://auth.example.net/?CAT=0oRLogEmBEZhYmMxMjOgWGOkAW9jZG4uZXhhbXBsZS5jb2
  0GGmVT8QoZATiiAYIAcGF1dGguZXhhbXBsZS5uZXQCggBhLxkBQIF4KGh0dHBzOi8vY29udGVu
  dC5leGFtcGxlLm5ldC9zb21lL2NvbnRlbnRYQKdEuQvf417HIXevB921vjz-TKIfbh0vjuWyZY
  yhM9LPR2s2S4xWbEXl_1crbWqfv5dTu2-mRI9ds1B8_hGHyGw
```

And the token is a CAT signed with abc123 (see Annex B for key material) that has these claims:

```
{
  / iss / 1: "cdn.example.com",
  / iat / 6: 1700000010,
  / catu / 312: { 1: [0, "auth.example.net"], 2: [0, "/"] },
  / catifdata / 320: ["https://content.example.net/some/content"]
}
```

A more complete example of this workflow in practice is the first Functional Use Case: Access an Asset with Renewal (section A.2).

4.9.2 Common Access Token Renewal (catr) claim

This claim is an instruction to the recipient that they MAY renew this token. The type of this claim is map with integer keys and values of varied types. The keys define the renewal parameter labels. The values are the renewal parameters. Recipients MAY consider tokens with too many elements in a "catr" map or array value to be invalid. Recipients MUST process tokens that contain fewer than 50 members of the "catr" map and fewer than 50 members of the "catr" value arrays. Recipients MUST NOT renew an unacceptable token. Recipients MUST NOT process a "catr" claim that renews a token if the "catr" claim is in an unacceptable claim set.

Although a token MUST NOT be renewed under this claim if it is unacceptable, a recipient also does not perform any additional validation of the client or its identity before performing the renewal. Simply validating the token itself is sufficient. A "catr" claim allows a client to extend their access by redeeming the token while it remains valid. It does not allow a client to obtain a valid token if they do not already have one. The "catif" claim is useful for some workflows that require that.

The Claim Key 323 is used to identify this claim.

Recipients MAY support this claim.

The "catr" claim is described by the following CDDL:

```
$$Claims-Set-Claims //= (catr-label => catr-value)
catr-label = 323
catr-value = {
      renewal-type-label         => renewal-type-value,
      renewal-expadd-label       => renewal-expadd-value,
      ? renewal-deadline-label ^ => renewal-deadline-label,
      ? renewal-name-label     ^ => renewal-name-value,
      ? renewal-params-label   ^ => renewal-params-label,
      ? renewal-code-label     ^ => renewal-code-label,
      * int                      => any
}

renewal-type-label = 0
renewal-type-value = int

renewal-expadd-label = 1
renewal-expadd-value = number

renewal-deadline-label = 2
renewal-deadline-value = number

renewal-cookie-name-label = 3
renewal-cookie-name-value = tstr

renewal-header-name-label = 4
renewal-header-name-value = tstr
```

```
renewal-cookie-params-label = 5
renewal-cookie-params-value = [ *tstr ]

renewal-header-params-label = 6
renewal-header-params-value = [ *tstr ]

renewal-code-label = 7
renewal-code-value = int
```

The label under the "catr-value" is an integer that indicates the parameter being defined. The parameter labels defined in this document are:

- 0: Renewal Type.
- 1: Expiration Extension.
- 2: Renewal Deadline.
- 3: Name for Cookie.
- 4. Name for Header.
- 5: Additional Cookie Parameters.
- 6: Additional Header Parameters.
- 7: Status Code for Redirects.

The four renewal types supported by this document are:

- 0: Automatic renewal
- 1: Cookie renewal
- 2: Header renewal
- 3: Redirect renewal

Each of these is defined in more detail below, along with the parameters they accept. A recipient MUST NOT renew a token based on a "catr" claim that has parameters that are not appropriate for the renewal type.

The renewal type and expiration extension parameters MUST be present in a "catr" claim. The renewal deadline parameter MAY be present in any "catr" claim.

The expiration extension parameter is a number, which indicates the number of seconds to be added to the time at which the token is received to determine the expiration time of the renewed token. The renewed token MUST have an "exp" claim in the base claim set that expires a number of seconds equal to the expiration extension parameter added to the time that the base token was verified. If this token had an "exp" claim as a direct member of the token, it is replaced. Otherwise, a new "exp" claim is added to the renewal token. "exp" claims that exist inside composition claims are not modified.

Likewise, the renewed token SHOULD have "iss" and "iat" claims in the base claim set that identify the issuer and issuance time of the token. The issuer is usually the recipient, but whichever entity issued the renewed token is the one to be identified. A renewed token MUST NOT contain an "iss" claim in the base claim that does not identify the issuer of the renewed token. The header of the renewed token SHOULD contain a "kid" parameter that identifies the key id of the key that signed the token.

The "cti" claim for the base claim set in a renewed token MUST NOT match the previous token. If the base token contains a "cti" claim, the renewed token MUST contain a "cti" claim with a value generated in a way unlikely to conflict with other "cti" claim values. If the base token does not contain a "cti" claim in the base claim set, the renewed token MUST NOT contain a "cti" claim in the base claim set.

If the recipient has a signing key suitable for token renewal, it MAY issue a new token. The new token MUST be signed by a key the recipient expects to be valid for that token until its new expiration. The signing key MAY be a key acceptable to only the recipient or it MAY be a key acceptable to other recipients as well. The selection of the renewal key and its management is outside the scope of this document.

The new token MUST have the same claims as the received token, except as noted above for the "exp", "iss", and "iat" claims. This includes claims not defined in this document. If a claim's definition provides for special handling on renewal, that special handling MUST be followed. Other than these claims, there are no claims in this document that define special handling for renewal.

If a renewal deadline is present, the token MUST also have an "exp" claim in the base claim of the token. If it does not, the recipient MUST NOT renew the token. If it does, the renewal deadline indicates the number of seconds before the expiration of the token, as defined in the "exp" claim in the base claim, that it SHOULD be renewed. A renewal claim processed between the deadline and the expiration SHOULD be renewed and it SHOULD NOT be renewed before then.

If a renewal deadline is not present, recipients MAY renew the token at whatever point they determine appropriate. The mechanism by which this is determined is out of scope for this document.

No future specification will define negative values for renewal types or parameter labels. These are reserved for private implementations.

4.9.2.1 Automatic renewal

When the type is 0, for automatic renewal, the renewed token is conveyed via the same mechanism in which it was received.

A token received in a cookie is processed as renewal type 1, for cookie renewal. If a name parameter is not provided, the name parameter MUST be treated as though it contained the name of the cookie in which the token was found.

A token received in a header is processed as renewal type 2, for header renewal. If a name parameter is not provided, the name parameter MUST be treated as though it contained the name of the header in which the token was parsed, unless it was found in the Authorization header, in which case the normal default header field name of "CTA-Common-Access-Token" is used.

A token received in a URI is processed as renewal type 3, for redirect renewal.

A token with automatic renewal MUST not have an additional parameters parameter, but MAY have other parameters that are inappropriate to the type of renewal. For example, a token with automatic renewal conveyed in a cookie MAY have a status code parameter, in which case it is ignored when renewed in a cookie.

4.9.2.2 Cookie renewal

When the type is 1, for cookie renewal, the renewed token is conveyed to the client in a cookie. In addition to the three core parameters, the following parameters are appropriate:

- 3: Name for Cookie.
- 5: Additional Cookie Parameters

The additional parameters array may have any number of additional members. These members are text strings parameters that MUST be added to the cookie. The new token MUST be set in a cookie of a name the recipient recognizes for requests. If the name parameter is provided, the new token MUST be set in a cookie of that name.

The additional parameter members MUST be added to the cookie. Recipients MUST validate that the parameters do not contain illegal characters for parameters as defined by RFC 6265 Section 5.2. Recipients SHOULD validate that parameters are valid for the Set-Cookie header as defined by RFC 6265. If the additional parameter members cannot properly or safely be added to the cookie, the recipient MUST NOT renew the token.

If, in the absence of a "catr" claim for cookie renewal, a recipient would have issued a Set-Cookie header for a cookie of the same name as it would issue for the renewal token, it MUST NOT renew the token.

4.9.2.3 Header renewal

When the type is 2, for header renewal, the renewed token is conveyed to the client in a header field. In addition to the three core parameters, the following parameters are appropriate:

- 4: Name for Header
- 6: Additional Header Parameters

The Additional Header Parameters parameter is an array of text strings that are parameters to be added to the token.

The new token will be encoded as the only element in a list of strings as defined by RFC 8941. The string will contain the token. Parameters described by the Additional Header Parameters parameter MUST be added associated to the token as defined by RFC 8941 Section 3.1.2. Recipients MUST ensure that the parameters contain no characters that are invalid according to RFC 8941.

The value of the name parameter is the name of a header with the name specified by the Name for Header parameter. If the recipient would include a header in the response in the absence of token renewal, the recipient MUST NOT renew the token using header renewal with that name. If the Name for Header parameter is absent, recipients MUST treat it as though it were provided with the value "CTA-Common-Access-Token".

4.9.2.4 Redirect renewal

When the type is 3, for redirect renewal, the renewed token is conveyed to the client by redirecting the client to a new URI with the token replaced with the new one. In addition to the three core parameters, the following parameter is appropriate:

- 7: Status Code for Redirects

Redirect renewal is only applicable when the token is conveyed in the URI. If the token is not in the URI, a redirect renewal claim in that token MUST NOT be processed.

The encoded token will replace the current token in the URI and that URI will be provided in the Location header field value of the response. The status code provided by the parameter, which MUST be between 300 and 399, inclusive, is the status code of the response. If no status code parameter is provided, recipients MUST treat it as though it were provided with the value 307.

Issuers should be aware that redirect renewal will be of limited value in situations where the client is making requests for different resources over time or when it does not cache or reuse the redirected URI. It is most useful when a client is repeatedly requesting the same resource and is prepared to cache redirections locally.

4.9.2.5 Other renewals

If the renewal type is unknown, the recipient MUST ignore it. An unknown renewal type is not a reason for rejection by itself.

4.9.2.6 Inside composition claims

If a "catr" claim is a member of a claim that contains other claims, it MUST NOT be processed unless no other claim in the composition would ordinarily result in the rejection of the claim set.

If multiple "catr" claims would be processed and result in renewal, the order in which they are processed is arbitrary. Since a given "catr" claim will not overwrite a cookie or header renewal with a given name, some "catr" claims may have no effect.

4.10 Security considerations

As the CAT is a CWT, all the considerations in RFC 8392 apply here, too. Issuers should be cautious as always to protect their encryption keys, since these can be used to forge tokens.

Recipients MUST ensure that resource limits for any limited resources like space or time are in place when processing tokens, particularly when processing resource intensive claims like regular expressions. Recipients MUST NOT trust issuers to provide non-malicious tokens. Recipients may find it useful to log information about requests that were rejected because their tokens exceeded the acceptable resource limits in the same ways they log other errors for later review. The details of this mechanism are out of the scope of this document, though.

The "catpor" claim is of particular note because it provides a mechanism by which issuers can ask recipients to "remember" identifiers for a time. Recipients MUST have limits on the number of identifiers to remember to avoid allowing issuers to consume excessive space. Additionally, these limits MUST be separate for different issuers. If the block lists for different issuers are stored in the same buffer, one issuer can potentially "flush out" blocked "catpor" claims from a different issuer. In this context, the issuer is the party issuing the token, which may or may not contain an "iss" claim documenting it.

4.10.1 Algorithms

Issuers MUST use asymmetric encryption algorithms when they do not have an established trust relationship with recipients that allows those recipients to also act as issuers on their behalf. Symmetric algorithms allow the recipient to act as an issuer as well as a recipient.

The list of acceptable algorithms may change over time. Recipients SHOULD accept any algorithm they consider to be secure in the context of the token. They SHOULD NOT fail to support an algorithm simply because it is not one of the mandatory minimum algorithms. Still, recipients will need to balance computational cost, library support, and the security parameters of the algorithms.

The mechanisms by which issuers and recipients exchange secrets and keys are not described in this document. Since the security of the token relies on the security of the secrets, sensitive keys need to be transferred securely.

4.10.2 Claims

Issuers have very wide discretion to select claims suitable for the task at hand. No claims are required in any particular token. Nevertheless, some claims SHOULD be used in every token unless there is a good reason to avoid them:

- exp: Nearly all tokens should expire. If they do not, the only thing that can invalidate them is communicating to recipients that they should not trust the key they were signed with or, where supported, revocation using the value in the "cti" claim.
- iss: This claim allows recipients to quickly locate verification keys in a shared environment.
- catu: Tokens should be tailored to allow access to only the resources required. This rarely means "everything".

Tokens that are renewable SHOULD have a non-renewable expiration that puts a maximum limit on the lifespan of renewal.

Recipients in a shared environment MUST ensure that CATs are signed with a key from an issuer that has the appropriate rights to grant access to a resource. It is not sufficient to ensure that they are signed with a key the recipient trusts, it must be a key associated with an issuer associated with the resource being requested. The details of how this is accomplished are outside the scope of this document, but it is a REQUIREMENT for a secure implementation.

4.10.3 Cache-Control

Recipients MUST ensure that the Cache-Control requirements in the Caching section (section 4.3.1.5.1) are followed. If content is served on account of a valid CAT and it is not marked private to prevent shared caches from serving it to others, the protection of the token will be ineffective.

4.10.4 Content

Issuers should additionally understand that the CAT only restricts access to the content via the URI that it protects; nothing prevents a bearer from using a valid CAT to retrieve content and then share that content itself. In situations where the content itself is to be protected, other techniques will be required. The CAT is not Digital Rights Management, though it can be used in combination with it.

Likewise, Issuers should be very careful when selecting and composing claims to ensure that they provide access only to the content they should. In particular, path parameters and query parameters are often ignored by origins that do not understand them, which can lead to inadvertent access. For example, a token that grants access to paths that end in "public/content.m3u8" will grant access to "https://example.com/secret;x=public/content.m3u8". If an origin ignores the path parameter x, it might grant access to content the issuer did not intend. Likewise, if an origin ignores superfluous or

duplicate query arguments, a requirement that the query argument contain "version=public" would be satisfied by a URI of "https://example.com/content?version=secret&x=version=public".

4.11 Privacy considerations

The claims of a CAT are not encrypted and can be read by the bearer and any other party on path with the issuer and the bearer and the bearer and the recipient. Furthermore, CATs in the URI are very likely to be logged as a matter of course by services processing requests. This means that it is very important that any privacy-sensitive information be encrypted. This can be accomplished by encrypting the entire token.

Several pieces of privacy-sensitive information MUST be encrypted, as noted in their descriptions. But in addition to those, any additional claims that contain sensitive information MUST be encrypted. To enforce this, recipients MAY consider as invalid any token that contains information that should be protected but is not. The definition of what is considered privacy-sensitive will change over time as technology and society changes. Recipients SHOULD publish lists of what they consider to be privacy-sensitive.

The "sub" claim bears special mention, since its purpose is to identify the bearer. It should usually be encrypted. However, the "sub" claim is informational and cannot be the basis for acceptance or rejection of a claim set. This means that recipients do not need a decryption key. This can reduce the scope of privacy loss when identities must be encoded in tokens. Issuers should not use an identifying "sub" claim unless it is necessary.

The "catif" claim is also worth special consideration. A "catif" token can cause privacy-sensitive information, including the URI, to be returned to the client in a Location header or other header that might cause a client to transmit that information to others. Issuers should take care to ensure that any redirections or retransmissions occur only within the same privacy context as the original request.

Recipients should be particularly sensitive to the contents of the token when renewal is being performed. If a shared cache serves a token to others, any poorly protected information in the token will be shared with unauthorized recipients. Strictly enforcing the Cache-Control requirements in the Caching section (section 4.3.1.5.1) will reduce this risk. Likewise, the above requirement that privacy-sensitive information be encrypted will reduce the risk of that information being revealed in the event that an improperly configured shared cache does distribute a response that contains a renewed token.

ANNEX A FUNCTIONAL USE CASES – INFORMATIVE

A.1 Introduction

This section describes some specific use cases supported by the specified mechanism and token format defined in this document.

At high level, a primary goal of the CAT is to support access tokens for accessing audiovisual content that is either Live or Video On Demand (VOD), taking into account that content can also be advertising that is either pre-, post- or mid-roll with the main content. Nevertheless, all these previous cases are similar in terms of token management: It is about getting a token valid for some part of content but irrelevant for others. In addition, there can be restrictions for accessing content (geo restriction for example). Tokens may need to be renewed. Some important use cases are then

- Accessing an asset with renewal of the token;
- Accessing an asset from multiple CDNs with the same token;
- Accessing restrictions based on one claim, or a mix of claims present in the token (for example, geolocation for blackouts); and
- Preventing sharing tokens between devices.

For clarity, the following use cases do not explicitly consider encrypted content, but content can be DRM-protected. In this case, the DRM and access token mechanisms are not interfering.

It is assumed in these examples that content is delivered in an HTTP Adaptive Streaming (HAS) format, for example DASH or HLS, but in practice, tokens can protect a variety of content delivered over HTTP.

These examples also use the keys defined in Annex B for signing.

These examples also demonstrate how "and" and "or" claims might work. This document does not define or normatively use these claims. It may be defined differently or not at all in another document. See Section 3.1 for details.

A.2 Access an asset with renewal of the token

For the first use case, a high-level view of the workflow for getting a token and subsequently receiving the media content is shown in Figure 1. While the Content Portal provides an initial access token, the token has an expiration date and at some point, it cannot be validated anymore. The CDN renews the access token possibly with some validation from the Content Portal that initially created the token and authorized the device.

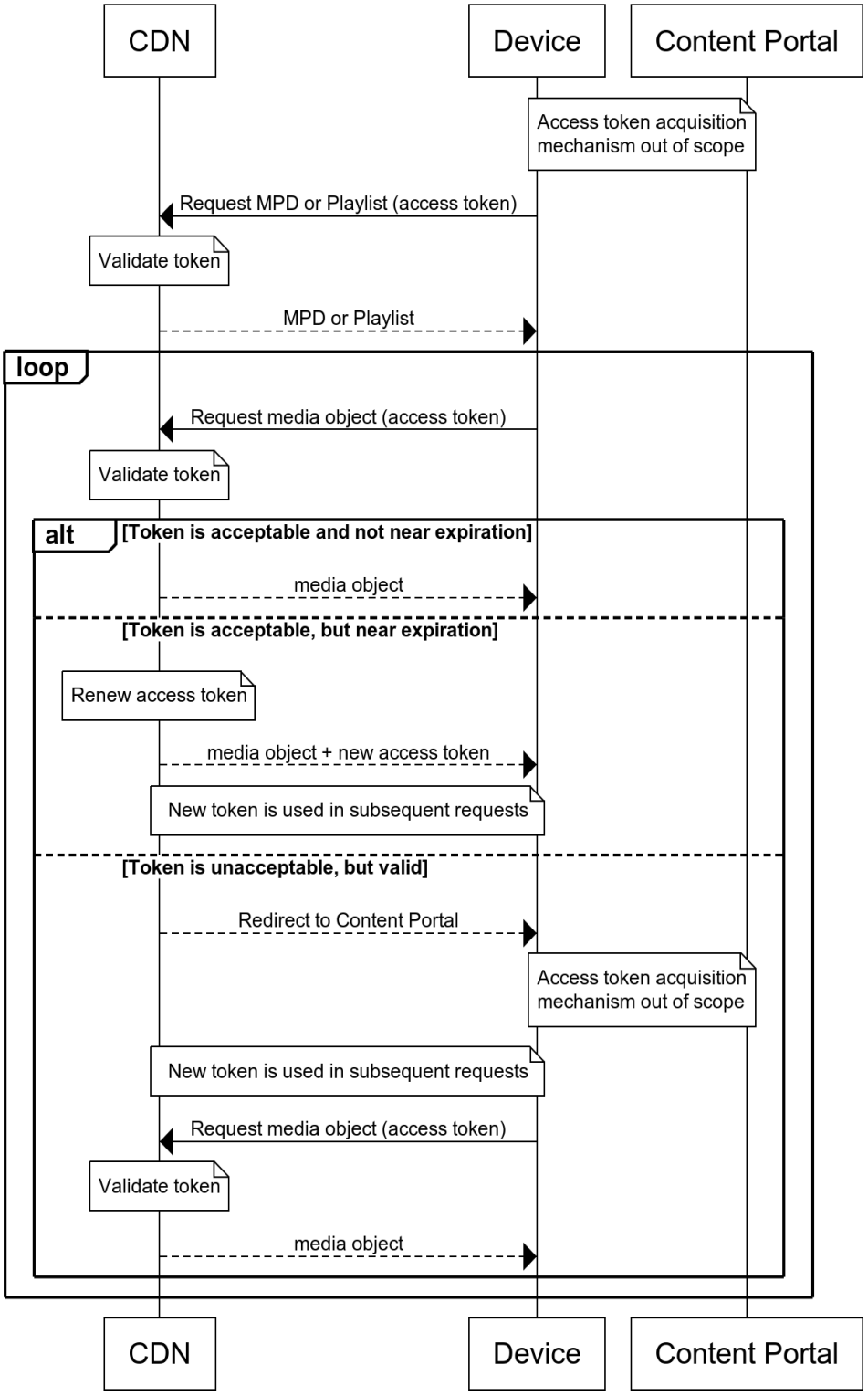

Figure 1: High-level workflow for the access of an asset with renewal of the token use case

This use case can be accomplished with a token conveyed via the Cookie header and using the "exp" and "catu" claims (sections 4.6.3 and 4.6.11, respectively):

```
{
	/iss/ 1: "brand.example.com",
	/exp/ 4: 1700000000,
	/catu/ 312: { /path/ 3: { /prefix/ 1: "/path/to/content/" } },
	/catr/ 323: {
		/type/ 0: /cookie/ 1,
		/expadd/ 1: 300,
		/deadline/ 2: 30,
		/cookie-name/ 3: "CTA-Common-Access-Token",
		/additional-cookie-params/ 5: [
			"Path=/path/to/content/",
			"HttpOnly",
			"Secure"
		]
	},
	/catif/ 322: {
		/exp/ 4: [ 307,
			{
				"Location": [
					"https://brand.example.com/auth?return=",
					/URI/ 0
				]
			}
		]
	},
	/and/ TBD_AND: [
		{
			/exp/ 4: 1700003600,
			/catif/ 322: {
				/exp/ 4: [ 307,
					{
						"Location": [
						"https://brand.example.com/auth?return=",
							/URI/ 0
						]
					}
				]
			},
		}
	]
}
```

This token allows access to all content in the "/path/to/content/" folder until November 14th, 2023, 22:13:20 GMT (which is 1700000000 unix time).

Suppose for this example, the token is used to request content at:

`https://brand.cdn.example.net/path/to/content/frag1.ts`

When this token is used at 1699999971, just after the renewal deadline, the CDN might respond with the content as usual, but add a Set-Cookie header for "CTA-Common-Access-Token" with a token that looks like this:

```
{
      /iss/ 1: "cdn.example.net",
      /exp/ 4: 1700000271,
      /catu/ 312: { /path/ 3: { /prefix/ 1: "/path/to/content/" } },
      /catr/ 323: {
            /type/ 0: /cookie/ 1,
            /expadd/ 1: 300,
            /deadline/ 2: 30,
            /cookie-name/ 3: "CTA-Common-Access-Token",
            /additional-cookie-params/ 5: [
                  "Path=/path/to/content/",
                  "HttpOnly",
                  "Secure"
            ]
      },
      /catif/ 322: {
            /exp/ 4: [ 307,
                  {
                        "Location": [
                              "https://brand.example.com/auth?return=",
                              /URI/ 0
                        ]
                  }
            ]
      },
      /and/ TBD_AND: [
            {
                  /exp/ 4: 1700003600,
                  /catif/ 322: {
                        /exp/ 4: [ 307,
                              {
                                    "Location": [
                                    "https://brand.example.com/auth?return=",
                                          /URI/ 0
                                    ]
                              }
                        ]
                  }
            }
      ]
}
```

Encoded, this might look like:

```
Set-Cookie: CTA-Common-Access-Token=0oRLogEmBEZhYmMxMjOgWQEDpgCBogQaZVP_EBkB
  QqEEghkBM6FoTG9jYXRpb26CeCZodHRwczovL2JyYW5kLmV4YW1wbGUuY29tL2F1dGg_cmV0dX
  JuPQABb2Nkbi5leGFtcGxlLm5ldAQaZVPyDxkBOKEDoQFxL3BhdGgvdG8vY29udGVudC8ZAUKh
  BIIZATOhaExvY2F0aW9ugngmaHR0cHM6Ly9icmFuZC5leGFtcGxlLmNvbS9hdXRoP3JldHVybj
  0AGQFDpQABARkBLAIYHgN3Q1RBLUNvbW1vbi1BY2Nlc3MtVG9rZW4Fg3ZQYXRoPS9wYXRoL3Rv
  L2NvbnRlbnQvaEh0dHBPbmx5ZlNlY3VyZVhABX5b19mxcMlEdyB5xik90bJuiavGRQwFn2oJxD
  uanl46FzA7XXnX05u89l11Cvn1gcumjENKJDCZCd54_VBsmg;
  Path=/path/to/content/; HttpOnly; Secure
```

(Linebreaks for readability, not present in the header. See Annex B for the keys.)

If the client then presents this new token on subsequent requests, they will have continued access with the extended expiration.

If, however, the client stops requesting content for a while (for example, while playback is paused) and then resumes after 1700000000, the token will be expired. In this case, the CDN can use the "catif" claim to redirect the client to an authentication server. It would return a response that looks like this:

```
HTTP/1.1 307
Cache-Control: private
Location: https://brand.example.com/auth?return=aHR0cHM6Ly9icmFuZC5jZG4uZXhh
  bXBsZS5uZXQvcGF0aC90by9jb250ZW50L2ZyYWcxLnRzCg
```

(return carriage and indent in Location field are not present on the wire, they are here for formatting only)

This would cause the client to redirect to the auth server, which validates the client in an unspecified way. This could be interactive, non-interactive, based on other information the auth server has, or any other mechanism suitable for the purpose. Once the auth server has determined the identity of the client, it uses the information passed to it to redirect the client back to the original location, where they can continue to request content.

Likewise, the token has an "and" claim that contains an "exp" claim. This "exp" claim is not in the base claimset, and so it does NOT get extended by token renewal. This creates a "long expiration" beyond which the token cannot be renewed by the recipient. Depending on the definition of the as-yet-undefined "and" claim, however, it might be the basis for a "catif" claim inside the "and" claim that directs the client to renew the token via redirect.

A.3 Very simple expiring link

At the other end of complexity is the simplest token that controls access to a single path prefix:

```
{
      /exp/ 4: 1700000000,
      /catu/ 312: {
            /path/ 3: { /prefix/ 1: "/b04c80bf74ff02b4/" }
      }
}
```

Encoded using the example ES256 asymmetric key (see Annex B), this could be conveyed in a URI via path parameter like this:

```
https://cdn.example.com/0oRDoQEmoFghogQaZVPxABkBOKEDoQFyL2IwNGM4MGJmNzRmZjAy
  YjQvWEDVGGAUXf_umBNaCwvXTr6vu9zqIuK-N3Cn8AqMq7gJD9PE9V2NyR7Nswch-Y7tY6dDf6
  gCKGuUEoJXt1PfNudF/manifest.m3u8
```

Encoded using the similar-strength example RSA key and asymmetric PS256 algorithm (see Annex B), this could be conveyed in the same place like this:

```
https://cdn.example.com/0oREoQE4JKBYIaIEGmVT8QAZATihA6EBci9iMDRjODBiZjc0ZmYw
  MmI0L1kBgAHDw7K6jWc72GXVM9Y0qkShGYZsT7fD6dW3wRKfd7NGM7_ZLizUT3PWOkbpHIOUlK
  M8XzfFrMO1bfldedskhlb556s2mRNUMwUl-dGYf6yEpcgRsX4vyR1qBuZropgm34-uAot-cdJ2
  rYibprOITlgVIFskFv_6egRLnPjoEF0hgMBLVAnnP8vqAb_OLGL3FR8WdxLdCom3X1ZpAysYqB
```

```
MceTC-jdvbx9rNP8mKihlfMQx8zS57tRZNAuLBznPlNEuJjS4kZoIe8URDUYkL9YxU-oU2TX4l
j3XSjv7OUaUvAyDNMen2zKCIFzs1m8wE1h1kz2b7JM0E2oFNGy1pBufQI3yjXBjrbwsKhum9ti
HbiHeg70abLqWw6PRAVX2agI-YmQ1Fx_c3VZ6GksuUZvd4wOaRYzKSQAhlL7M26mktza6PQxyA
5p5ANZrzymK0b4CYWEmgv07reJB_ocEBQ4SU9zEWudcqZle4DUgMRpLU-a-mZ5BAX84do5PTqs
BNlQ/manifest.m3u8
```

And encoded using the example symmetric secret with the HMAC256 algorithm (see Annex B), it would look like this:

```
https://cdn.example.com/0YRDoQEFoFghogQaZVPxABkBOKEDoQFyL2IwNGM4MGJmNzRmZjAy
YjQvWCDrzYuMLoWS2FuuX2tHdGlm3EZczlfhU3_lNeALlWhE6Q/manifest.m3u8
```

All of these example tokens are encoded minimally, choosing to omit the key id ("kid") in the header and the issuer ("iss") claim in the claim set, so recipients will need to know which key to use for validation based on the content requested. This can be specified via CDN configuration, for example.

The token can also be put in different places. For example, the token can be in a query parameter as well:

```
https://cdn.example.com/b04c80bf74ff02b4/manifest.m3u8?CAT=0YRDoQEFoFghogQaZ
VPxABkBOKEDoQFyL2IwNGM4MGJmNzRmZjAyYjQvWCDrzYuMLoWS2FuuX2tHdGlm3EZczlfhU3_
lNeALlWhE6Q
```

Or, if the path were longer, it might be on a later path parameter:

```
https://cdn.example.com/b04c80bf74ff02b4/content;CAT=0YRDoQEFoFghogQaZVPxABk
BOKEDoQFyL2IwNGM4MGJmNzRmZjAyYjQvWCDrzYuMLoWS2FuuX2tHdGlm3EZczlfhU3_lNeALl
WhE6Q/manifest.m3u8
```

It could also be in a header:

```
GET                  /b04c80bf74ff02b4/content/manifest.m3u8                HTTP/1.1
Host:                                                                  cdn.example.com
CTA-Common-Access-Token:            0YRDoQEFoFghogQaZVPxABkBOKEDoQFyL2IwNGM4MGJmNzRmZjA
yYjQvWCDrzYuMLoWS2FuuX2tHdGlm3EZczlfhU3_lNeALlWhE6Q
```

(Newlines and whitespace within the access token is for visual purposes only. It is transmitted all as one line.)

Or it could be in any of the other valid places for a token.

A.4 Access an asset from multiples CDNs with the same token

The second use case is shown in Figure 2. In this case, the token provided by the Content Portal allows playing back the content from multiple CDNs. Each CDN is able to validate the token and possibly to issue a new token when it has expired. A token has an expiration date and when renewed by one CDN, it allows the device to request segments on another CDN.

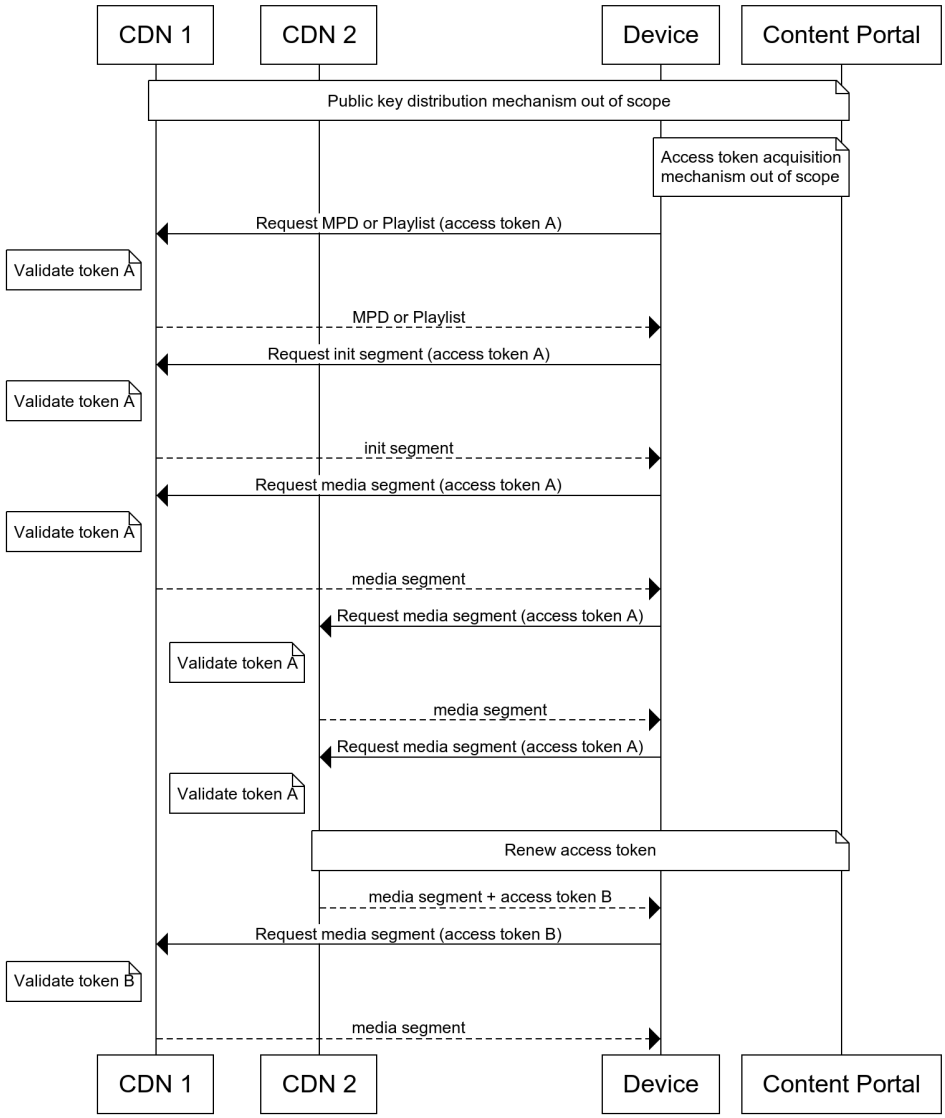

Figure 2: High-level workflow for the access of an asset from multiple CDNs with the same token use case

In order for this to work, each CDN must trust the signing keys of the other CDNs. This is most appropriately done by having each CDN publish the public keys they are using for that content provider, along with the public key of the Content Portal. The mechanism is out of scope, but if both CDNs trust the public keys of the other two parties, they can each validate the tokens of the others.

This can also be accomplished by having all three parties use the same symmetric key. This comes with additional management complexities in order to maintain security, so it is not recommended.

A.5 Access restrictions based on the IP of the requester

This use case is one where the token is bound to the IP of the requester. The token may look like this:

```
{
     /iss/ 1: "brand.example.com",
```

```
      /exp/ 4: 1700000000,
      /catu/ 312: { /path/ 3: { /prefix/ 1: "/path/to/content/" } },
      /catnip/ 311: [/IPv4/ 52([16, /10.0.*.*/ h'0a00'])]
}
```

This token is valid until its expiration for content in the described path, as in prior use cases. This token, however, is only acceptable when the client makes the request from an IP address in the 10.0.0.0/16 network.

Because the prefix length in the token is not longer than 24 bits, the "catnip" claim is not automatically required to be encrypted.

If the client makes a request from a different IP address, perhaps because it is a mobile device in motion and has switched networks, the token will be unacceptable and access will be denied.

A.6 Preventing token reuse

The use case is described in Figure 3. In this case, the token provided by the Content Portal is reused by either the same device or a second device. It might be reused either to access the same URI or a different URI. Since the token is single-use, once it is used, it cannot be redeemed again.

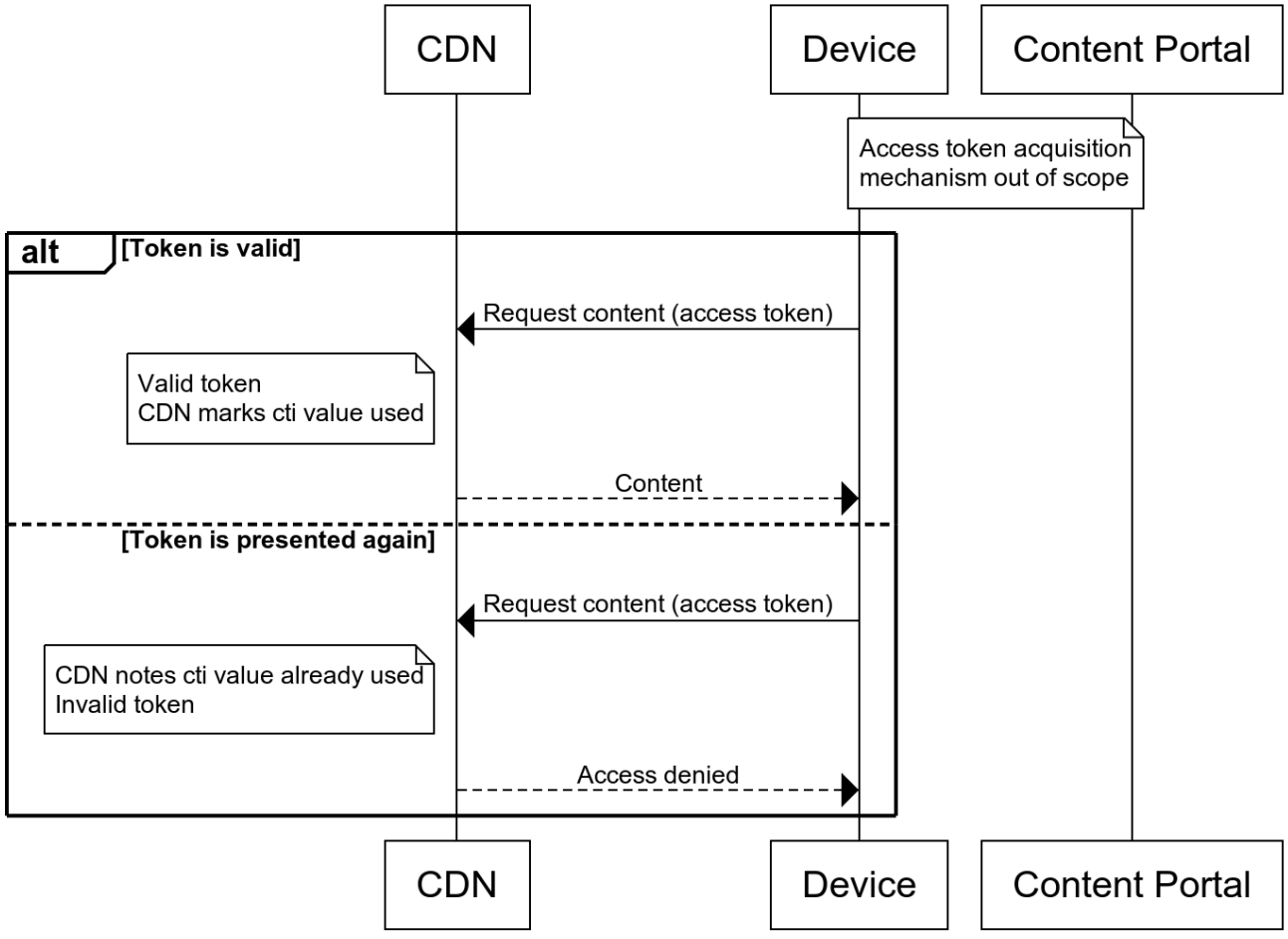

Figure 3: High-level workflow for the use case of preventing token reuse

This is accomplished with a CWT ID ("cti") claim.

```
 {
      /iss/ 1: "brand.example.com",
      /exp/ 4: 1700000000,
      /catu/ 312: { /path/ 3: { /prefix/ 1: "/path/to/content/" } },
      /cti/ 7: h'94b58b7b3858b65ff654c96bd8c8575c',
      /catreplay/ 308: 1
}
```

This token is valid until its expiration, allows access to a path prefix of "/path/to/content" and the CDN can deny access to anyone attempting to reuse it. In a token, it might look like this:

```
http://brand.cdn.example.com/path/to/content/manifest.m3u8?CAT=0oRLogEmBEZhY
  mMxMjOgWEmlAXFicmFuZC5leGFtcGxlLmNvbQQaZVPxAAdQlLWLezhYtl_2VMlr2MhXXBkBNAE
  ZATihA6EBcS9wYXRoL3RvL2NvbnRlbnQvWEC56L_dPH6a2E5ks2SrAsUZ8ISYzqTISyChjMUlD
  MiST_rfkc3Kl7RLwoYjs5vk-GAkvRNQEwv9oNMf36ib19x4
```

A.7 Restricting access to a specific geolocation by geohash

The flow for this use case is shown in Figure 4. In this case, the token contains claims that restrict access to content based on some defined rules. As a simple example, as shown in the flow, it could be a restriction on the geo-location (country based for example). More complex cases are possible, such as a restriction based on a mix of claims.

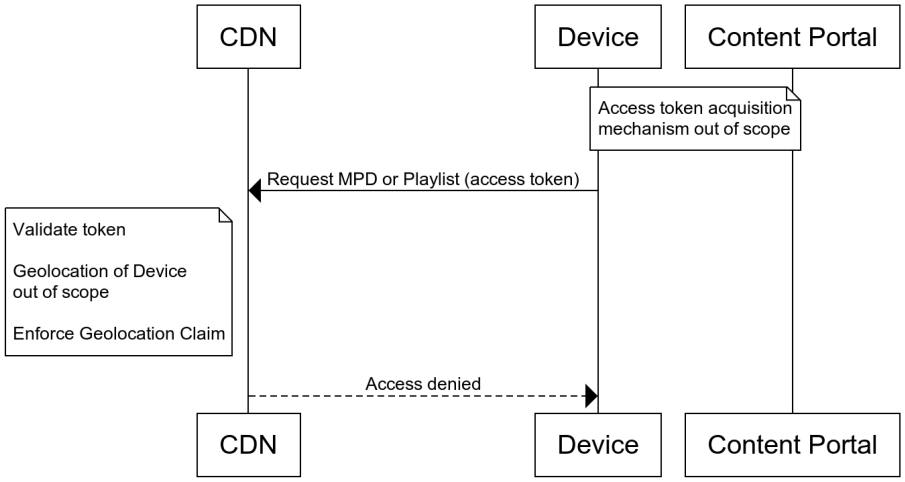

Figure 4: High-level workflow for the Live main asset

The token for this might look like this:

```
 {
      /iss/ 1: "brand.example.com",
      /exp/ 4: 1700000000,
      /catu/ 312: { /path/ 3: { /prefix/ 1: "/path/to/content/" } },
      /geohash/ 282: "9vc0"
}
```

This token allows access to all content in the "/path/to/content/" folder until November 14th, 2023, 22:13:20 GMT (which is 1700000000 unix time) from locations the CDN determines are within that Geohash (which is roughly Abilene, Texas, USA and the surrounding area).

How the CDN determines the location of the user is out of scope and may in many cases be very difficult.

Signed with the example ES256 asymmetric key, this token might look like this:

```
0oRLogEmBEZhYmMxMjOgWDukAXFicmFuZC5leGFtcGxlLmNvbQQaZVPxABkBGmQ5dmMwGQE4oQOh
  AXEvcGF0aC90by9jb250ZW50L1hAq6lBYR95F2QEkbO0Dif4hbD4FfyCz0k81FPA9Y_05BRRkY
  lvPsQDiHw0dXfybXGlxJY1o0M2qR69Uzu7xBTyxA
```

A.8 Restricting access to a specific geolocation by region

A user can access the resource only if they are from an allowed geographical location. The restriction is expressed in a claim within the token. There can be a list of allowed or disallowed geographical locations. Figure 5 gives a high-level view of the flow.

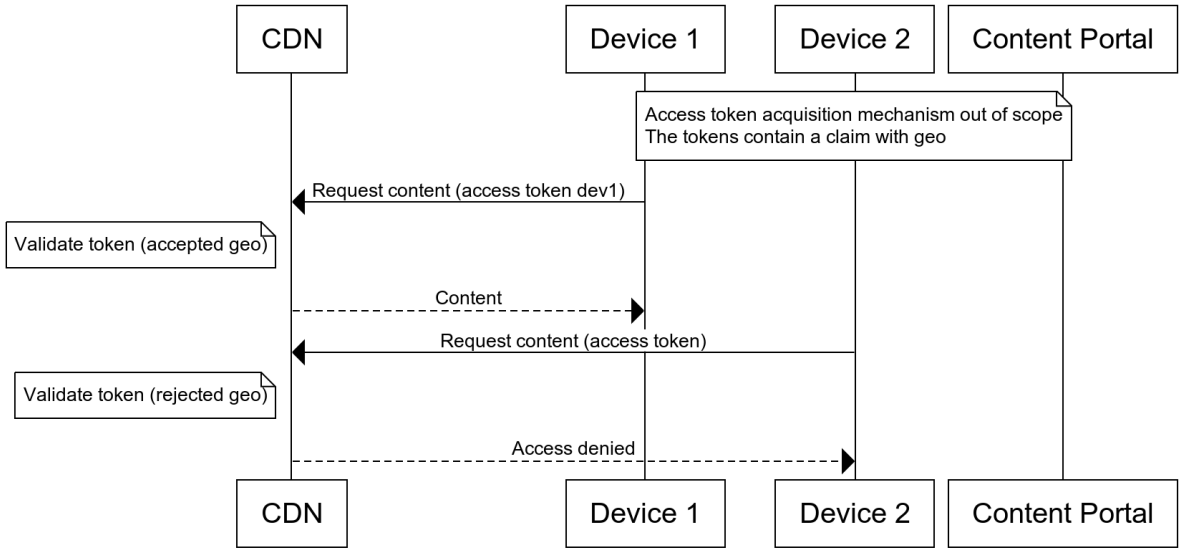

Figure 5: High-level workflow for restricting the access for a specific geography

This token might look like this:

```
 {
      /iss/ 1: "brand.example.com",
      /exp/ 4: 1700000000,
      /catu/ 312: { /path/ 3: { /prefix/ 1: "/FU1BpBusguM/" } },
      /or/ TBD_OR: [
            { /catu/ 312: { /ext/ 8: { /exact/ 0: ".mpd" } } },
            { /catu/ 312: { /ext/ 8: { /exact/ 0: ".mp4" } } }
      ],
      /cti/ 7: h'32ce93be7c4b912c0c74f96c6f1f0199',
      /catgeoiso3166/ 316: ["fr"]
}
```

This token is valid until its expiration, for paths that start with "/FU1BpBusguM/", have either ".mpd" or ".mp4" extensions, and are requested from a location in France.

In a URI, it might look like this:

```
https://edge.cdn.example.com/FU1BpBusguM/frag01.ts?CAT=0oRLogEmBEZhYmMxMjOgW
  GSmAIKhGQE4oQihAGQubXBkoRkBOKEIoQBkLm1wNAFxYnJhbmQuZXhhbXBsZS5jb20EGmVT8QA
  HUDLOk758S5EsDHT5bG8fAZkZATihA6EBbS9GVTFCcEJ1c2d1TS8ZATyBYmZyWEAQL5_HdaYfO
  -0Nu1-OdlAJezk-jzzWiWpfZYtua7JBNzZ3IDGocXrrAw43PJxDXKD_nd4VaWj0FgPMbSYPdtF
  O
```

If this token is requested from a location outside of France, content can be denied.

A.9 Bypassing geolocation restrictions

This is a continuation of the previous use case where a resource is only available from a specific geographic location, for cases like testing. These other claims could allow access from a specific ASN, for example, even if they do not meet the geographic requirement.

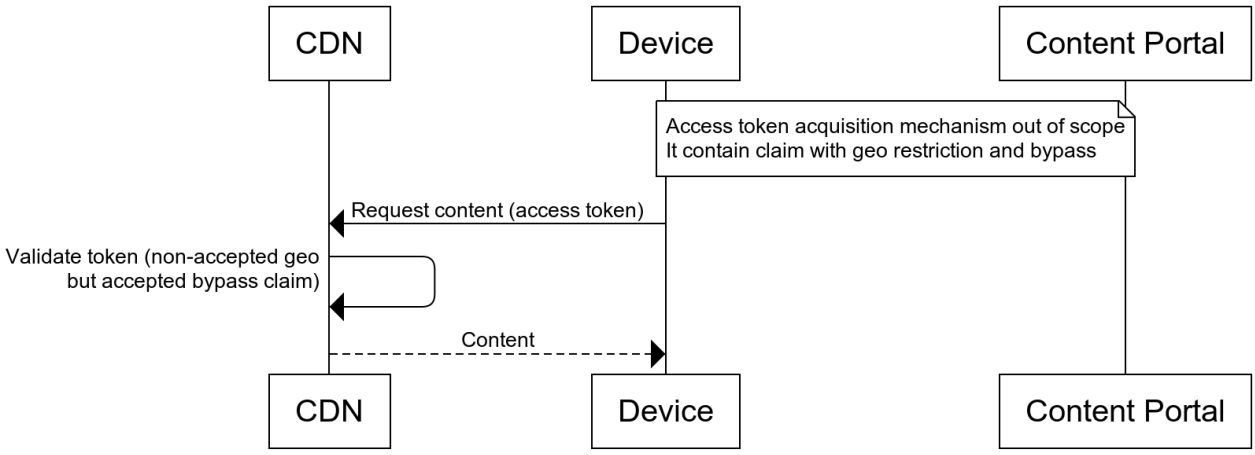

Figure 6: High-level workflow for bypassing geolocation restrictions

The previous example token could be expanded with an additional bypass claim:

```
{
      /iss/ 1: "brand.example.com",
      /exp/ 4: 1700000000,
      /cti/ 7: h'32ce93be7c4b912c0c74f96c6f1f0199',
      /and/ TBD_AND: [
            {
                  /catu/ 312: { /path/ 3: { /prefix/ 1: "/FU1BpBusguM/" } },
                  /or/ TBD_OR: [
                        { /catu/ 312: { /ext/ 8: { /exact/ 0: ".mpd" } } },
                        { /catu/ 312: { /ext/ 8: { /exact/ 0: ".mp4" } } }
                  ]
            },
            {
                  /or/ TBD_OR: [
                        { /catgeoiso3166/ 316: ["fr"] },
                        { /catnip/ 311: 64496 }
```

```
                ]
            }
        ]
}
```

So, the token is valid in the same conditions as the previous example, but if the request is made from ASN 64496, the token is valid outside of France.

A.10 Restricting access to a resource to a specific time window

A user can access the resource only if they are doing it in a specified time window. The restriction is expressed in a claim within the token. Figure 7 gives a high-level view of the flow.

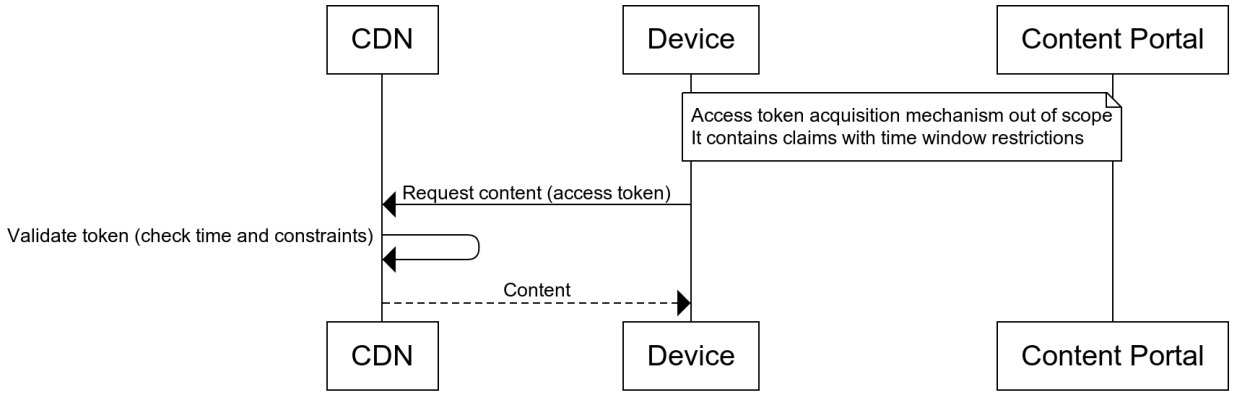

Figure 7: High-level workflow for restricting the access to a specific time window

This kind of claim can be accomplished by using both an "nbf" and "exp" claim:

```
{
      /iss/ 1: "brand.example.com",
      /exp/ 4: 1700000000,
      /nbf/ 5: 1699999700,
      /catu/ 312: { /path/ 3: { /prefix/ 1: "/FU1BpBusguM/" } }
}
```

This claim is valid for only 5 minutes, from November 14, 2023, 22:08:20 GMT to November 14, 2023, 22:13:20 GMT. It can be issued ahead of time to ensure that the client is ready to receive the content as soon as it becomes available. It allows access only to content in the "/FU1BpBusguM/" directory.

In a URI, the token might look like this:

```
https://cdn.example.com/FU1BpBusguM;CAT=0oRLogEmBEZhYmMxMjOgWDWkAXFicmFuZC5leGFtcGxlLmN
vbQQaZVPxAAUaZVPv1BkBDqEDoQFtL0ZVMUJwQnVzZ3VNL1hAPFaS6qDgT4zGA1R8JiSV-
cyAoDsq4fQBIobGYFvOck6ACSuZUFdsVY7EKC2te54Fw78r1-suLsL0hG6a07Wt1Q/timed.m3u8
```

A.11 Varying expiration based on file type

A user can access content with a single token that has multiple expirations that depend on the content being requested.

```
{
      /iss/ 1: "brand.example.com",
      /catu/ 312: { /path/ 3: { /prefix/ 1: "/FU1BpBusguM/" } },
```

```
	/or/ TBD_OR: [
		{
			/catu/ 312: { /extension/ 8: { /exact/ 0: ".mpd" } },
			/exp/ 4: 1700000060
		},
		{
			/catu/ 312: { /extension/ 8: { /exact/ 0: ".mp4" } },
			/exp/ 4: 1700003600
		}
	]
}
```

This allows the user to redeem the token until Unix time 1700000060 if the extension matches ".mpd" and 1700003600 if the extension matches ".mp4". It also only provides access to content in the "/FU1BpBusguM/" directory. If the extension does not match one of those two options, access may be denied on account of the "or" claim.

In a URI, it might look like this:

```
https://cdn.example.com/FU1BpBusguM/content.mpd?CAT=0oRLogEmBEZhYmMxMjOgWFGj
  AIKiBBplU_E8GQE4oQihAGQubXBkogQaZVP_EBkBOKEIoQBkLm1wNAFxYnJhbmQuZXhhbXBsZS
  5jb20ZATihA6EBbS9GVTFCcEJ1c2d1TS9YQC6kHctu3bCAN1KA1sk7of5AQ1KX_hn_pgiqBBau
  _53XosOruWuqSiRwvwpNMQ7jXE35GVtCsD07C_bMLZyUBd4
```

This approach has limitations, however. For example, because the "catr" claim only creates or extends an "exp" claim in the base claimset, the "exp" claims in the composition cannot be extended.

A.12 Varying expiration based on file type with renewal

As a variation on the previous example, a user can renew a token of one file type, but not for another.

```
{
	/catu/ 312: { /path/ 3: { /prefix/ 1: "/FU1BpBusguM/" } },
	/exp/ 4: 1700000060,
	/or/ TBD_OR: [
		{
			/catu/ 312: { /extension/ 8: { /exact/ 0: ".mpd" } },
			/exp/ 4: 1700000060
		},
		{
			/catu/ 312: { /extension/ 8: { /exact/ 0: ".mp4" } }
		}
	],
	/catr/ 323: {
		/type/ 0: /cookie/ 1,
		/expadd/ 1: 300,
		/deadline/ 2: 60,
		/cookie-name/ 3: "CTA-Common-Access-Token",
		/additional-cookie-params/ 5: [
			"Path=/FU1BpBusguM/",
			"HttpOnly",
			"Secure"
```

```
            ]
      }
}
```

This controls access to the "/FU1BpBusguM/" directory as before. The token itself may effectively expire at Unix time 1700000060 because the "exp" claim in both the base claimset and also the option in the "or" composition claim have the same expiration.

However, if the token is renewed, the base "exp" claim in the base claimset will be extended by 300 seconds. At this point, a request for a ".mp4" file will be controlled by the "exp" claim in the base claimset because there is no "exp" claim in the subclaim. A request for a ".mpd" file will be controlled by both the "exp" claim in the base claimset and also the "exp" claim in the first option in the "or" claim. Since the base claimset "exp" claim will always be larger, the sooner claim will control and requests for ".mpd" files will be rejected after Unix time 1700000060, even with a renewed token. (Note: The "or" claim is not defined or normatively used in this document. It may be defined differently or not at all in a future document. See Section 3.1 for details.)

In this way, renewal can operate usefully for one file type and not another. This approach may be useful for some VOD workflows.

In a URI, it might look like this:

```
https://cdn.example.com/FU1BpBusguM/content.mpd?CAT=0oRLogEmBEZhYmMxMjOgWImk
  AIKiBBplU_E8GQE4oQihAGQubXBkoRkBOKEIoQBkLm1wNAQaZVPxPBkBOKEDoQFtL0ZVMUJwQn
  VzZ3VNLxkBQ6UAAQEZASwCGDwDd0NUQS1Db21tb24tQWNjZXNzLVRva2VuBYNyUGF0aD0vRlUx
  QnBCdXNndU0vaEh0dHBPbmx5ZlNlY3VyZVhAIuma19ahcYvZ8Z6u1_7VxZ5P5pxj2y4HNgUJUR
  9KoPW4VOLL2G3J_pemhDh3RRSEdK5HdoncLFZoqpSSbhBg6Q
```

A.13 Combined Common Access Token and Watermarking Token

Because both the CAT and the DASH-IF Forensic A/B Watermarking Token are CBOR Web Tokens, a single token can satisfy both specifications. For example:

```
{
      /exp/ 4: 1700000000,
      /catu/ 312: {
            /path/ 3: { /prefix/ 1: "/b04c80bf74ff02/" }
      },
      /wmver/ 300: 1,
      /wmvnd/ 301: 245,
      /wmpatlen/ 302: 32,
      /wmpattern/ 303: 0x0a0b0c0d
}
```

In practice, the "wmpattern" ought to be encrypted, but it's presented here in the clear for simplicity. This token is a CAT that expires and limits access to a specific directory. It is also a watermarking token that defines specific processing for the content that is returned as described in the watermarking specification.

```
https://cdn.example.com//content.m3u8?CAT=0oRLogEmBEZhYmMxMjOgWDWmBBplU_EAGQ
  EsARkBLRj1GQEuGCAZAS8aCgsMDRkBOKEDoQFwL2IwNGM4MGJmNzRmZjAyL1hA_OVBlXaB3pE0
  Ws6grw-RYONXzyERIy3BLGNQEDMVwNwqYX2Cad1YMBqVHHYBx40GY0j4wXT2hDOWEmK-5yREjg
  %3D%3D
```

Note that because this token complies with both the Watermarking and CAT specifications, it uses padding on the end. And because this is conveyed in the value field of a query arg, the padding MUST be URL Encoded.

ANNEX B KEYS FOR EXAMPLES – INFORMATIVE

These examples use these example keys for signing. Implementations should not use these keys for production implementations, as the private and public keys are both published in this document. However, in order to allow the validation of these examples, the keys are published here.

B.1 Asymmetric keys

Most examples in this document using asymmetric encryption use an ES256 key with the private key:

```
-----BEGIN EC PRIVATE KEY-----
MHQCAQEEILwS9vH0DU3Q4EbP2ehnxNAYs0qIpmf/VrLe2S0BN0JSoAcGBSuBBAAK
oUQDQgAECwP++YTSZeYhCIOodY2U2fylTqd7b9jWptatWIXTe25VYGv8bRYv0Ojq
2UkNlL/MeOz/DzAFDZ6VDD2JF8LzHA==
-----END EC PRIVATE KEY-----
```

And public key:

```
-----BEGIN PUBLIC KEY-----
MFYwEAYHKoZIzj0CAQYFK4EEAAoDQgAECwP++YTSZeYhCIOodY2U2fylTqd7b9jW
ptatWIXTe25VYGv8bRYv0Ojq2UkNlL/MeOz/DzAFDZ6VDD2JF8LzHA==
-----END PUBLIC KEY-----
```

These are identified by the key id ("kid") of "abc123".

B.2 RSA key

For examples that use RSA, this key is used:

```
-----BEGIN PRIVATE KEY-----
MIIG/QIBADANBgkqhkiG9w0BAQEFAASCBucwggbjAgEAAoIBgQCRs+FCY5CC+noU
58XkDC7zcCztUTY9NuJ6cuLsjzZbpdRHhusmERTlkHuKEmwbPBVmmf5knGZZCY+S
4YCYk9gKUxcO+SQDGXOJUAzzHtPXtZRyqm5BhoeAFrz/uDLltlp8JCF8DLuyATlu
/usSgJv6TDCcS3t+GrRdZ3dzKPfF2G7Wulnn/fWJ8qASOAIkzdZFKL/omYVXohYs
Kyo0dz+kX5wAoy/bLcA/j+qXmIbEWI2w2ZOo35VWy4zG+J3JemWyckMPgvmKMgIr
Tmkr7lfAt4oUIewNjAXT8rRUGxVerMZ1+ibEaVWFOFi37Nx1JuLqCPFKi+k1F8Hu
ca5bHPWEfoTUnlTRUm+XRnQ4YIrW5TPBcCeDiYSvT1ECS6QVGnqQWZSYOnHLAHKu
180QEKRa2eWRKSb0hUPOVofpvIEjLjfgdJ6bnDA8EO9DWoZ4E8LjhW1XUma9DEPM
kLt7sxCjocvlcQeppdY+FxiShfg7lbZOjGU525MR6KZjGEJr8V8CAwEAAQKCAYAR
7oMfn49Of13SVYAGWUrXTQv1HRSWzhAWJS4xEk23U8kKEIcU2pn+yBxcLs49P9TG
btdfMdxqwtbNAJHKCrqrnUmY3ygDyvhsLXzv2EXBe33MlZXCMg4FFpymFYO2FQ49
JFk2oik2IW5xxE/RVCCzhPhKlAObzn+PgJyz1evrM9WC+dD3e0ricl4plS5hBzAt
```

```
Yh7UP+ioPgY1OLlrVu3Ei2ZwsaTBzcymRfkHio8Iz86IYGQK2ZF7QCE676gY/sYh
HkLIvt5Y379k/OBet3LYe44EaMIoK89QNRmh4PLM5qq1gBDxB94N+fZX1R4TgwK1
5zG34kpVtzK1E+spFRW3OiBLdQfmQEvYFlUqS8JP//VXH0xoAmXAOeIZRQwdWBY7
9XFWel8HIND/7gIQzNZWlHFDzWwJYMe6j0MFJ3J30fnglZsW9pelm+2jd+MtANFt
95WNJCPGDHYFLLjtU/o80eYhZNcwa5ORPDrq8AoqsvMzZAJHpiI8JtG1KH1QzWkC
gcEAxx/G597n8EqjEMB81QEj63luct25Z90xvP+9o9wJKXZiTCvdDIxUH+++eiwU
sp0l/TgsUDV6/ZuP/B95hWgv7PXMVcAfl/g7rMHruwbQEg6RX78xVyYjt+XzNqib
oUgTqadMmuNgxpu6SZrm8ZjevHMWMwTXiWruGIkUpiqQYY4k4I/G5R31ON+kgtWp
bhk6H/h11Ds6uD1R4CgN8el4pXoRRXSTvCnKZiSZzWOnUx4SyXjTKyGYGZokzo/j
wN59AoHBALtR2nEpGLVa/WhOt/Na4wbGdJYUfyGICb1IU6S6Qoo/tAvXHmFTj2yV
SRcEKoYVCRTU2BfYL2S/93PZ5/5V4JI3qaTnGB7dZnGYZOERWCFZsmsGBxzMvYJx
LLCMQMcOux13w+9MoRk4L42x5AfBjR6tqE9RoPPtTb5MZnscIrH5ZhCt6XVU2shX
AwTQ9ElMkSkVtIeDGXSuvONGroKNzL1ZdKwkrhOh2+QkQgE6koAtpYpT9Y+g5ja/
P85gfW+KCwKBwQC2wnS2ZsGsq2RlQLaSbfyA6LA130mHid5X4MY2+gKyuoRX8z5s
gPPJaCsESScqFNBETGVnrN5YnIoX1u+bQVgTfjpwTejxe5WR516sBwG8QQCux3XK
ziekZvrGpRRKgDQD15iY5K7zOwVM/06OWke84UhzxTLqXmlvvYllRdXEpEk2jt/3
dHqZG7MLnn1mnuLZaxsCkuY/KZDXUwwRh78C6jml7OyXnCe3fkpn19pAkG6f2jS+
9pN6lB7KNAjPAaUCgcB0bsx4yjKxh307+LIecb/r1i9hmhvrm8NyBCL5oUz64d3n
/i7EEEx2EdT8mXpVHX4nlKU9LQNIfIozWJ77WCXevPqVpY84oCWtNix+OWmuUTK1
k2mYXSruiapdktAw6tcj2e3QMLFyG5pzWL+EkqpFB2w24WZKOJa6UoLgwvxN49et
Rmx2oefb3znhJICzjwQrpXIxSSvpezOtkaGnRLyedZwv5BzP1yv3AxKD8QRe6ACd
mtJoFHsETJw8i31sd0ECgcBiJvhm1rIaGNyufbspsZb3etgu23IKE6oneuLm0l0y
YVoLk77ay41wx/xD76boXF8o5R/ShNF/VPNhz+12XRzOrsl8q6wtYKF0VUgs6qal
YC3MZB6lqunHc0eHma43hfrFFR9chLX4ySc/tZlaBvd43O+iq44oK0JjoCIYNcM5
6sIwAH3L6Lb6xul3YJ16ZDF+BnOtbOe0dxs4S/SXzCsIk8gDV4/7q1Ih7vN3IyYl
B7PzSlN37r0j/85usgXcfs0=
-----END PRIVATE KEY-----
```

And the associated public key:

```
-----BEGIN PUBLIC KEY-----
MIIBojANBgkqhkiG9w0BAQEFAAOCAY8AMIIBigKCAYEAkbPhQmOQgvp6FOfF5Awu
83As7VE2PTbienLi7I82W6XUR4brJhEU5ZB7ihJsGzwVZpn+ZJxmWQmPkuGAmJPY
ClMXDvkkAxlziVAM8x7T17WUcqpuQYaHgBa8/7gy5bZafCQhfAy7sgE5bv7rEoCb
+kwwnEt7fhq0XWd3cyj3xdhu1rpZ5/31ifKgEjgCJM3WRSi/6JmFV6IWLCsqNHc/
pF+cAKMv2y3AP4/ql5iGxFiNsNmTqN+VVsuMxvidyXplsnJDD4L5ijICK05pK+5X
```

```
wLeKFCHsDYwF0/K0VBsVXqzGdfomxGlVhThYt+zcdSbi6gjxSovpNRfB7nGuWxz1
hH6E1J5U0VJvl0Z0OGCK1uUzwXAng4mEr09RAkukFRp6kFmUmDpxywByrtfNEBCk
WtnlkSkm9IVDzlaH6byBIy434HSem5wwPBDvQ1qGeBPC44VtV1JmvQxDzJC7e7MQ
o6HL5XEHqaXWPhcYkoX4O5W2ToxlOduTEeimYxhCa/FfAgMBAAE=
-----END PUBLIC KEY-----
```

These are identified by the key id ("kid") of "def456". This is a 3072-bit key, which at the time of writing provides the same 128-bits of security as the ES256 key above.

B.3 Symmetric key

Examples using MAC (which requires both the issuer and recipient to have the same key) use an HMAC 256/256 key:

```
i2p+my+r4Cne2k/1E/Isz6yP/RajEXBAtr+kMlbuZOc=
```

This key is base64-encoded for display.

This is identified by the key id ("kid") of "xyzzy".

ANNEX C OUT-OF-BAND INFORMATION – INFORMATIVE

In order for issuers and recipients to interoperate, they must communicate at some level. In addition to the core information that must be communicated in order for a recipient to act as a proxy in general, the CAT requires exactly two mandatory pieces of information to be communicated out-of-band from the issuer to the recipient:

- The issuer name; and
- A key that can be used to verify token signatures.

However, there are other pieces of information required for optional claims and functionality.

C.1 From the issuer

C.1.1 Encrypted claims

If the issuer is putting data in an encrypted claim that is not merely informational, it will need to communicate a decryption key to the recipient.

C.1.2 Revocation

The "cti" claim values to be revoked must be communicated to the recipient. Useful implementations of this are likely to need timely communication and processing.

C.1.3 Renewal

The selection of a renewal key for a "catr" claim may need to be communicated out-of-band. Renewal can be done using a key known only to the recipient and not valid for any other recipient or it can be done using a key communicated to it by the issuer. Either way, this selection must be communicated to the recipient.

Likewise, renewal is done at the discretion of the recipient, so if the recipient has specific requirements that must be met in order to perform renewal, they should be communicated. In particular, recipients should communicate which renewal methods they support, if any.

C.1.4 If

If the issuer intends to use a "catif" claim to do special processing of failed validations, it may need a shared secret with the recipients that allows the recipient to sign those access tokens.

C.1.5 Processing priority

If the issuer intends some configuration settings on a recipient to take priority over the actions prescribed or proscribed herein, that will need to be communicated. Recipients can be configured to provide access in ways other than by a CAT or to deny access regardless of the presentation of an acceptable CAT, so if any such configurations exist, they will need to be communicated in whatever way is appropriate to them.

Issuers should be aware that some behaviors that are controllable via a CAT may be controllable via alternate mechanisms or configuration as well.

C.1.6 Scope

If the issuer intends for only a subset of the content they are authoritative for to require an acceptable token for it to be served, it needs to communicate which content it should apply to. The details of this are out of scope for this document, but recipients need to know which content should require a token.

For requests that do not require a token for access, but are configured for additional processing, like removing a token from the URI before making upstream requests, the secondary parsing may still need to be applied. This is an independent part of the configuration of a recipient.

C.2 From the recipient

The primary information a recipient needs to communicate is about what it supports. No information is mandatory, since issuers can assume a recipient supports only the mandatory claims and nothing else. But a recipient that intends to communicate either a non-conforming implementation or support for optional claims can do so by publishing a list of supported claims.

A recipient also needs to agree with the issuer on the names of the parameters in which tokens will be accepted if tokens are to be found in path parameters, query parameters, or cookies. Similarly, the recipient needs to agree with the issuer on whether an Authorization header with a Bearer or DPoP scheme should be processed as a token. And if a non-standard header field is to be checked for a CAT, that must be agreed upon as well. Of course, any of these can potentially be configuration items, in which case the issuer might furnish them to the recipient instead.

Likewise, recipients should publish lists of supported algorithms for MACing and signing (and the associated verifications) and encryption and decryption.

If recipients require issuers to supply a "kid" header field, they should indicate that.

If recipients support the SHA-256 and SHA-512/256 matching types, they should indicate that.

If recipients support "cnf" claim methods other than "jkt", they should also publish those.

A recipient that supports revocation via the value in the "cti" claim should indicate so, possibly by way of describing the mechanism by which revoked "cti" claim values are communicated.

A recipient that supports Coordinate Reference Systems other than WGS84 should list the supported CRSs.

A recipient that supports DPoP settings for the "catdpop" claim should publish the list of settings they support. This document does not define any additional settings, but future documents may.

ANNEX D CDDL – NORMATIVE

```
; Core Elements
cwt-claims = {
  * $$Claims-Set-Claims
  * label => any
}

label = int / tstr
label-or-labelset = label / [ + label ]

stringoruri = tstr   ; Interpreted per RFC 8392 Section 2
numericdate = number ; Interpreted per RFC 8392 Section 2

; Issuer claim
$$Claims-Set-Claims //= (iss-label => iss-value)
iss-label = 1
iss-value = stringoruri

; Audience claim
$$Claims-Set-Claims //= (aud-label => aud-value)
aud-label = 3
aud-value = stringoruri / [ * stringoruri ]

; Expiration claim
$$Claims-Set-Claims //= (exp-label => exp-value)
exp-label = 4
exp-value = numericdate

; Not Before claim
$$Claims-Set-Claims //= (nbf-label => nbf-value)
nbf-label = 5
nbf-value = numericdate

; CWT ID claim
$$Claims-Set-Claims //= (cti-label => cti-value)
```

```
cti-label = 7
cti-value = bstr

; Common Access Token Replay claim
$$Claims-Set-Claims //= (catreplay-label => catreplay-value)
catreplay-label = 308
catreplay-value = $catreplay-value
$catreplay-value /= 0 ; permitted
$catreplay-value /= 1 ; prohibited
$catreplay-value /= 2 ; reuse-detection
$catreplay-value /= uint .gt 2 ; undefined

; Common Access Token Probability of Rejection claim
$$Claims-Set-Claims //= (catpor-label => catpor-value)
catpor-label = 309
catpor-value = [catpor-probability, catpor-id, ?catpor-exp]
catpor-probability = number
catpor-id = label / bstr
catpor-exp = numericdate

; Common Access Token Version claim
$$Claims-Set-Claims //= (catv-label => catv-value)
catv-label = 310
catv-value = 1

; Common Access Token Network IP claim
$$Claims-Set-Claims //= (catnip-label => catnip-value)
catnip-label = 311
catnip-value = [ + catnip-object ]

; Autonomous System Number (ASN), IPv6 address or IPv4 address
catnip-object = uint / ipv6-address-xor-prefix / ipv4-address-xor-prefix

; IP addresses as defined in in [RFC9164] section 5
; excluding the ipv6-address-with-prefix option
```

```
ipv6-address-xor-prefix = #6.54(ipv6-address / ipv6-prefix)
ipv4-address-xor-prefix = #6.52(ipv4-address / ipv4-prefix)

ipv6-address = bytes .size 16
ipv4-address = bytes .size 4

ipv6-prefix-length = 0..128
ipv4-prefix-length = 0..32

ipv6-prefix-bytes = bytes .size (uint .le 16)
ipv4-prefix-bytes = bytes .size (uint .le 4)

ipv6-prefix = [ipv6-prefix-length, ipv6-prefix-bytes]
ipv4-prefix = [ipv4-prefix-length, ipv4-prefix-bytes]

; Common Access Token URI claim
$$Claims-Set-Claims //= (catu-label => catu-value)
catu-label = 312
catu-value = {
  * uripart => match
}

uripart = &(
  scheme: 0,
  host: 1,
  port: 2,
  path: 3,
  query: 4,
  parent-path: 5,
  filename: 6,
  stem: 7,
  extension: 8,
)

match = {
```

```
  ? exact-match ^ => tstr,
  ? prefix-match ^ => tstr,
  ? suffix-match ^ => tstr,
  ? contains-match ^ => tstr,
  ? regex-match ^ => [ tstr, * tstr ],
  ? sha256-match ^ => bstr,
  ? sha512-256-match ^ => bstr
}

exact-match = 0
prefix-match = 1
suffix-match = 2
contains-match = 3
regex-match = 4
sha256-match = -1
sha512-256-match = -2

; Common Access Token Method claim
$$Claims-Set-Claims //= (catm-label => catm-value)
catm-label = 313
catm-value = [ + tstr ]

; Common Access Token ALPN claim
$$Claims-Set-Claims //= (catalpn-label => catalpn-value)
catalpn-label = 314
catalpn-value = [ + bstr ]

; Common Access Token Header claim
$$Claims-Set-Claims //= (cath-label => cath-value)
cath-label = 315
cath-value = {
  * text => match ; match is defined in Section 4.6.10
}

; Common Access Token Geographic ISO3166 claim
```

```
$$Claims-Set-Claims //= (catgeoiso3166-label => catgeoiso3166-value)
catgeoiso3166-label = 316
catgeoiso3166-value = [ + tstr ]

; Common Access Token Coordinate claim
$$Claims-Set-Claims //= (catgeocoord-label => catgeocoord-value)
catgeocoord-label = 317
catgeocoord-value = [ + location ] / #6.279([crs: any, [ + location ]])
location = [ latitude: number, longitude: number, radius: number ]
     / #6.279([crs: any, [latitude: number, longitude: number, radius:
number ]])
; Tag 279 is the Coordinate Reference System Wrapper defined in [GEOHASH]

; Geohash claim
$$Claims-Set-Claims //= (geohash-label => geohash-value)
geohash-label = 282
geohash-value = untagged-geohash-value
  / #6.279([crsw-system, untagged-geohash-value])
untagged-geohash-value = geohash-string
  / [ * geohash-string ]
geohash-string = tstr
  / #6.279([crsw-system, tstr])
crsw-system = any ; interpreted as tag 104 when untagged

; Common Access Token Altitude claim
$$Claims-Set-Claims //= (catgeoalt-label => catgeoalt-value)
catgeoalt-label = 318
catgeoalt-value = altitude-spec / #6.279([crs: any, altitude-spec])
altitude-spec = [ altitude: number, deviation: number ]

; Common Access Token TLS Public Key claim
$$Claims-Set-Claims //= (cattpk-label => cattpk-value)
cattpk-label = 319
cattpk-value = bstr
```

```
; Subject claim
$$Claims-Set-Claims //= (sub-label => sub-value)
sub-label = 2
sub-value = stringoruri

; Issued At claim
$$Claims-Set-Claims //= (iat-label => iat-value)
iat-label = 6
iat-value = numericdate

; Common Access Token If Data claim
$$Claims-Set-Claims //= (catifdata-label => catifdata-value)
catifdata-label = 320
catifdata-value = string / [ * string ]

; Confirmation claim
$$Claims-Set-Claims //= (cnf-label => cnf-value)
cnf-label = 8
cnf-value = {
  ? cnf-jkt-label ^ => cnf-jkt-value
  * label => any ; other values described in RFC8747
}

cnf-jkt-label = 323
cnf-jkt-value = bstr .size 32

; Common Access Token DPoP Settings claim
$$Claims-Set-Claims //= (catdpop-label => catdpop-value)
catdpop-label = 321
catdpop-value = {
  ? -1 ^ => [ *int ] ; crit
  ?  0 ^ => uint     ; window
  ?  1 ^ => uint     ; jti
  * int  => any
}
```

```
; Common Access Token If claim
$$Claims-Set-Claims //= (catif-label => catif-value)
catif-label = 322
catif-value = { * label-or-labelset => catif-action }
catif-action = [
  http-status: uint, ; RFC9110 Section 15
  ? headers: catif-headers,
  ? kid: tstr, ; RFC9052 Section 7.1
]
catif-headers = {
  * catif-header-name => catif-header-value
}
catif-header-name = tstr
catif-header-value = tstr / [ + catif-header-value-element ]
catif-header-value-element = tstr / int / cwt-nullable-claims
cwt-nullable-claims = { * label => any }

; Common Access Token Renewal claim
$$Claims-Set-Claims //= (catr-label => catr-value)
catr-label = 323
catr-value = {
  renewal-type-label         => renewal-type-value,
  renewal-expadd-label       => renewal-expadd-value,
  ? renewal-deadline-label ^ => renewal-deadline-label,
  ? renewal-name-label     ^ => renewal-name-value,
  ? renewal-params-label   ^ => renewal-params-label,
  ? renewal-code-label     ^ => renewal-code-label,
  * int                      => any
}

renewal-type-label = 0
renewal-type-value = int

renewal-expadd-label = 1
```

```
renewal-expadd-value = number

renewal-deadline-label = 2
renewal-deadline-value = number

renewal-cookie-name-label = 3
renewal-cookie-name-value = tstr

renewal-header-name-label = 4
renewal-header-name-value = tstr

renewal-cookie-params-label = 5
renewal-cookie-params-value = [ *tstr ]

renewal-header-params-label = 6
renewal-header-params-value = [ *tstr ]

renewal-code-label = 7
renewal-code-value = int
```

ANNEX E IANA CONSIDERATIONS – INFORMATIVE

E.1 HTTP fields

Registration of the following field is requested for the Hypertext Transfer Protocol (HTTP) Field Name Registry:

Field Name: CTA-Common-Access-Token
Status: Permanent
Specification Document: CTA-5007
See section 4.4.2 for more details around the use of this field.

E.2 CWT claims

Registration of the following claims is requested for the CBOR Web Token (CWT) Claims registry:

Claim Name: catreplay
Claim Description: Common Access Token Replay
JWT Claim Name: N/A
Claim Key: 308
Claim Value Type: unsigned integer
Change Controller: Consumer Technology Association
Specification Document: CTA-5007

Claim Name: catpor
Claim Description: Common Access Token Probability of Rejection
JWT Claim Name: N/A
Claim Key: 309
Claim Value Type: array
Change Controller: Consumer Technology Association
Specification Document: CTA-5007

Claim Name: catv
Claim Description: Common Access Token Version
JWT Claim Name: N/A
Claim Key: 310
Claim Value Type: unsigned integer
Change Controller: Consumer Technology Association
Specification Document: CTA-5007

Claim Name: catnip
Claim Description: Common Access Token Network IP
JWT Claim Name: N/A
Claim Key: 311
Claim Value Type: array

Change Controller: Consumer Technology Association
Specification Document: CTA-5007

Claim Name: catu
Claim Description: Common Access Token URI
JWT Claim Name: N/A
Claim Key: 312
Claim Value Type: map
Change Controller: Consumer Technology Association
Specification Document: CTA-5007

Claim Name: catm
Claim Description: Common Access Token Method
JWT Claim Name: N/A
Claim Key: 313
Claim Value Type: array
Change Controller: Consumer Technology Association
Specification Document: CTA-5007

Claim Name: catalpn
Claim Description: Common Access Token ALPN
JWT Claim Name: N/A
Claim Key: 314
Claim Value Type: array
Change Controller: Consumer Technology Association
Specification Document: CTA-5007

Claim Name: cath
Claim Description: Common Access Token Header
JWT Claim Name: N/A
Claim Key: 315
Claim Value Type: map
Change Controller: Consumer Technology Association
Specification Document: CTA-5007

Claim Name: catgeoiso3166
Claim Description: Common Access Token Geographic ISO3166
JWT Claim Name: N/A
Claim Key: 316
Claim Value Type: array
Change Controller: Consumer Technology Association
Specification Document: CTA-5007

Claim Name: catgeocoord
Claim Description: Common Access Token Geographic Coordinate
JWT Claim Name: N/A
Claim Key: 317
Claim Value Type: array
Change Controller: Consumer Technology Association
Specification Document: CTA-5007

Claim Name: catgeoalt
Claim Description: Common Access Token Geographic Altitude
JWT Claim Name: N/A
Claim Key: 318
Claim Value Type: array
Change Controller: Consumer Technology Association
Specification Document: CTA-5007

Claim Name: cattpk
Claim Description: Common Access Token TLS Public Key
JWT Claim Name: N/A
Claim Key: 319
Claim Value Type: byte string
Change Controller: Consumer Technology Association
Specification Document: CTA-5007

Claim Name: catifdata
Claim Description: Common Access Token If Data
JWT Claim Name: N/A
Claim Key: 320
Claim Value Type: string or array
Change Controller: Consumer Technology Association
Specification Document: CTA-5007

Claim Name: catdpop
Claim Description: Common Access Token DPoP Settings
JWT Claim Name: N/A
Claim Key: 321
Claim Value Type: map
Change Controller: Consumer Technology Association
Specification Document: CTA-5007

Claim Name: catif
Claim Description: Common Access Token If
JWT Claim Name: N/A

Claim Key: 322
Claim Value Type: map
Change Controller: Consumer Technology Association
Specification Document: CTA-5007

Claim Name: catr
Claim Description: Common Access Token Renewal
JWT Claim Name: N/A
Claim Key: 323
Claim Value Type: map
Change Controller: Consumer Technology Association
Specification Document: CTA-5007

E.3 CWT confirmation method

Registration of the following method is requested for the CWT Confirmation Methods registry:

Confirmation Method Name: jkt
Confirmation Method Description: JWK SHA-256 Thumbprint
JWT Confirmation Method Name: jkt
Confirmation Key: 323
Confirmation Value Type: byte string
Change Controller: Consumer Technology Association
Specification Document: CTA-5007


CTA™
Consumer Technology Association, producer of CES®

Consumer Technology Association Document Improvement Proposal

If in the review or use of this document a potential change is made evident for safety, health or technical reasons, please email your reason/rationale for the recommended change to standards@CTA.tech.

Consumer Technology Association
Technology & Standards Department
1919 S Eads Street, Arlington, VA 22202
FAX: (703) 907-7693
standards@CTA.tech